A PROJECT REPORT ON

SIMPLE MALWARE SCANNER USING YARA

Submitted By

SHAIK UBAIDUR RAHMAN

PROJECT ID

PTID-CSPP-JUL-26-511

BATCH ID

11-MAY-26-CSPP-BUN-750-WDM0930-HYD-MAD

DECLARATION

I hereby declare that the project report entitled **"Simple Malware Scanner Using YARA"** is my original work carried out as part of the phase 2 in Skillogic internship requirements for the completion of **CSPP program.**

The project involved developing and testing a simple malware scanning system using **YARA, PowerShell, and SHA-256 hashing** from a Windows environment. The project focused on rule-based malware detection, recursive file scanning, automated scanning, endpoint security scanning, threat hunting, incident-response support, and evidence collection.

I further declare that the work presented in this report has been carried out for educational and authorized laboratory purposes. The information, observations, results, screenshots, and analysis included in this report represent the work performed during the project.

I have not knowingly copied or reproduced the work of another person without appropriate acknowledgement.

Name:
SHAIK UBAIDUR RAHMAN

Project ID:
PTID-CSPP-JUL-26-511

Date:
6th Sept 2026

ABSTRACT

Malware is one of the major cybersecurity threats faced by individuals and organizations. Traditional security solutions primarily rely on predefined signatures and may not always provide sufficient flexibility for detecting customized or emerging suspicious files. This project presents a **Simple Malware Scanner Using YARA**, a lightweight and customizable rule-based malware detection system designed to identify potentially malicious files through predefined patterns and indicators.

The project uses **YARA** to create rules based on suspicious strings, command indicators, scripting indicators, and other file characteristics. A **PowerShell-based automation layer** is implemented to execute YARA scans, perform recursive directory scanning, count rule matches, measure scan duration, and generate detection reports.

The scanner was tested using controlled test samples containing predefined suspicious indicators. The implementation successfully detected multiple rule matches across individual files and nested directories while correctly allowing a clean test file to remain undetected. The scanner also supports endpoint security scanning of selected directories, threat-hunting activities, and basic incident-response evidence collection.

For incident-response purposes, detected files can be documented along with their associated YARA rule, file path, detection timestamp, and **SHA-256 hash**. Dedicated reports are generated to assist with analysis and documentation.

The final implementation demonstrates that YARA can provide a lightweight, flexible, and customizable approach to file-level malware detection. The project also provides practical exposure to YARA rule development, automated security scanning, threat hunting, endpoint monitoring, and incident-response concepts.

Keywords: YARA, Malware Detection, Cybersecurity, Threat Hunting, Incident Response, Rule-Based Detection, PowerShell, SHA-256, File Scanning.

TABLE OF CONTENTS

Chapter / Section	Title	Page No.
1	Title Page	1
2	Declaration	2
3	Abstract	3
4	Table of Contents	4
5	Introduction	5
6	Problem Statement	6
7	Objectives of the Project	8
8	Business Case	10
9	Existing System	15
10	Proposed System	17
11	Requirements Specification	21
12	Technology Overview	25
13	System Architecture	29
14	Project Directory Structure	34
15	YARA Rule Design and Development	38
16	System Implementation	43
17	Testing Methodology	50
18	Results and Analysis	57
19	Screenshots and Evidence	62
20	Advantages of the Proposed System	81
21	Limitations	86
22	Future Enhancements	90
23	Security Considerations	96
24	Conclusion	101
25	References	103
26	Appendix	105

5. INTRODUCTION

Malware is a significant cybersecurity threat that can affect individual systems, organizational networks, servers, and endpoints. Malicious or suspicious files may contain executable code, scripts, commands, embedded strings, or other characteristics that can be used to compromise systems, steal information, disrupt operations, or establish unauthorized access. As malware techniques continue to evolve, security professionals require flexible tools that can identify suspicious characteristics at the file level.

YARA is a pattern-matching and rule-based detection tool widely used in malware research, threat detection, digital forensics, and threat hunting. YARA allows security analysts to create rules that describe specific characteristics of files. These characteristics can include textual strings, hexadecimal patterns, file properties, and logical conditions. When a scanned file satisfies the conditions defined in a YARA rule, the file is reported as a potential match.

This project, titled **"Simple Malware Scanner Using YARA,"** focuses on developing a lightweight file-scanning solution that uses custom YARA rules to identify potentially suspicious files. The project combines YARA with **PowerShell automation** to provide a practical scanning workflow on a Windows environment.

The implemented scanner supports individual file scanning as well as recursive directory scanning. Custom rules were created to identify controlled test indicators associated with suspicious PowerShell activity, command execution, scripting activity, and other predefined patterns. A clean test file was also included to validate that files without matching indicators do not generate detections.

The project further extends the basic YARA scanning capability through endpoint scanning, threat-hunting functionality, and incident-response reporting. The endpoint scanner can inspect selected directories such as Downloads, Desktop, and Documents. The threat-hunting component recursively searches a designated directory for predefined indicators. The incident-response component records relevant information about detected files, including the triggered YARA rule, file path, detection timestamp, and SHA-256 hash.

During final testing, the main scanner successfully processed **6 controlled test files**, produced **5 rule matches**, and completed the scan in approximately **0.056 seconds**. One clean test file produced no detection, demonstrating the basic distinction between matching and non-matching files.

The project provides practical experience in rule-based malware detection, YARA rule development, automated security scanning, recursive file analysis, endpoint security, threat hunting, and incident-response evidence collection. It demonstrates how a lightweight and customizable YARA-based solution can complement existing security mechanisms by providing additional visibility into file-level indicators.

6. PROBLEM STATEMENT

The increasing number and complexity of malware threats create significant challenges for individuals, organizations, and cybersecurity professionals. Malicious files can use different techniques, obfuscation methods, scripts, commands, and file characteristics to avoid detection and compromise systems. Identifying suspicious files quickly is therefore an important part of an effective cybersecurity strategy.

Traditional antivirus solutions primarily depend on known signatures and other detection mechanisms. Although these solutions are useful for detecting established threats, they may not always provide the flexibility required for customized security investigations, malware research, threat hunting, and detection of organization-specific indicators.

The major problems addressed by this project are:

6.1 Difficulty in Detecting Customized Threats

Organizations may encounter malware or suspicious files that contain unique characteristics not covered by their existing security signatures. Security analysts need the ability to define customized detection patterns according to the requirements of a particular investigation.

6.2 Dependence on Predefined Detection Mechanisms

Traditional security solutions may rely heavily on vendor-maintained signatures. This can make it difficult for security teams to quickly create and deploy their own detection logic when investigating a new suspicious artifact.

6.3 Lack of Lightweight and Customizable Scanning

Security analysts may require a lightweight tool that can be executed against a selected file or directory without deploying a complete endpoint security platform. A rule-based scanner can provide a simple method for performing targeted file analysis.

6.4 Delayed Identification of Suspicious Files

Manual inspection of large numbers of files can be time-consuming. A scanner that automatically checks files against predefined rules can reduce the time required to identify potentially suspicious artifacts.

6.5 Limited File-Level Visibility

Security investigations often require analysts to understand why a file was considered suspicious. Identifying the specific rule or indicator that triggered a detection can provide useful information during analysis and investigation.

6.6 Challenges in Threat Hunting

Threat hunters may need to search directories for known indicators of compromise (IOCs). Performing this task manually across multiple directories can be inefficient. A recursive

YARA scan can assist analysts by automatically searching files and subdirectories for predefined indicators.

6.7 Need for Basic Incident-Response Evidence

When a suspicious file is identified, investigators need useful information about the artifact. Recording the file path, triggered detection rule, detection time, and SHA-256 hash provides basic evidence that can support further investigation.

Problem Definition

Therefore, the project addresses the need for a **simple, lightweight, customizable, and automated malware-scanning solution** that can use YARA rules to identify potentially suspicious files, perform recursive scanning, support endpoint and threat-hunting activities, and generate basic detection and incident-response reports.

7. OBJECTIVES OF THE PROJECT

The primary objective of this project is to design and implement a **Simple Malware Scanner Using YARA** capable of identifying potentially suspicious files through customizable rule-based detection.

The specific objectives of the project are as follows:

7.1 Develop a YARA-Based Malware Detection System

To implement a file-scanning solution that uses YARA rules to identify files containing predefined suspicious patterns, strings, and indicators.

7.2 Create Custom Detection Rules

To develop customized YARA rules for detecting controlled indicators associated with suspicious PowerShell activity, command execution, scripting activity, and other predefined malware-related patterns.

7.3 Implement Recursive File Scanning

To enable the scanner to search not only files in a specified directory but also files located within nested subdirectories.

7.4 Automate the Scanning Process

To use PowerShell to automate YARA execution, scanning, detection counting, result processing, and report generation.

7.5 Measure Scanning Performance

To measure the time required to complete a scan and demonstrate that the implemented scanner can perform lightweight and rapid file analysis.

7.6 Support Endpoint Security

To implement scanning of selected endpoint directories such as Downloads, Desktop, and Documents so that potentially suspicious files can be identified as an additional security layer.

7.7 Support Threat Hunting

To implement recursive scanning of a designated threat-hunting directory for predefined indicators of compromise (IOCs), helping demonstrate proactive security investigation.

7.8 Support Incident Response

To generate an incident-response report containing important information about detected files, including the triggered YARA rule, file path, detection timestamp, and SHA-256 hash.

7.9 Validate Detection Accuracy

To test the scanner using controlled suspicious test samples and clean files, verifying that matching files are detected while files without matching indicators are not reported.

7.10 Generate Security Reports

To automatically generate reports containing scan results and relevant evidence that can be reviewed during malware analysis, threat hunting, and incident-response activities.

7.11 Develop Practical Cybersecurity Skills

To gain practical experience in YARA rule development, malware detection, file analysis, PowerShell automation, endpoint security, threat hunting, and incident-response processes.

8. BUSINESS CASE

SIMPLE MALWARE SCANNER USING YARA

8.1 Introduction

YARA is a powerful pattern-matching tool widely used in malware research, threat detection, digital forensics, and threat hunting. It enables security professionals to create rules based on known characteristics of suspicious or malicious files, including textual strings, binary patterns, file structures, and other indicators.

The **Simple Malware Scanner Using YARA** developed in this project uses customized YARA rules to scan files and directories for predefined suspicious indicators. PowerShell automation is used to execute scans, process detection results, measure scan performance, and generate reports.

The project provides a practical approach to file-level malware detection and demonstrates how YARA can be incorporated into security analysis and investigation workflows.

8.2 Problem Statement

Malware continues to evolve and presents significant security risks to computer systems and organizations. Security teams require flexible mechanisms for identifying suspicious files and investigating potential threats.

The major challenges addressed by this project include:

- Difficulty in detecting customized or previously unidentified suspicious files.
- Dependence on predefined antivirus signatures and detection mechanisms.
- Lack of lightweight and customizable file-scanning solutions.
- Delayed identification of suspicious artifacts during security investigations.
- Limited visibility into file-level indicators.
- Difficulty in manually searching large directory structures for suspicious indicators.
- Need for basic evidence collection during incident-response activities.

These challenges can increase the risk of system compromise, data loss, operational disruption, and delayed incident response.

8.3 Solution Overview

The proposed solution is a **Simple Malware Scanner Using YARA** that scans files against predefined YARA rules.

The system consists of a YARA detection engine, custom rule files, controlled test samples, and PowerShell automation scripts.

The general workflow is:

```
Input Files / Directories
          ↓
     YARA Rules
          ↓
    YARA Scanner
          ↓
   Pattern Matching
          ↓
   Detection Results
          ↓
 PowerShell Processing
          ↓
 Reports / Evidence
```

The scanner supports individual file scanning and recursive directory scanning. When a file contains an indicator matching a YARA rule, the corresponding rule name and file path are reported.

Additional components provide endpoint scanning, threat hunting, and incident-response evidence collection.

8.4 Key Features and Benefits

8.4.1 Rule-Based Detection

The scanner uses YARA rules to identify files containing predefined suspicious patterns and indicators. Custom rules were created and tested as part of the project.

Benefit: Provides flexible file-level detection based on security analyst-defined conditions.

8.4.2 Lightweight and Fast

The scanner uses the YARA command-line engine with PowerShell automation and does not require a large security platform for the demonstrated scanning workflow.

During final testing, the scanner processed **6 controlled test files**, produced **5 rule matches**, and completed the scan in approximately **0.056 seconds**.

Benefit: Demonstrates that targeted file scanning can be performed rapidly with limited processing overhead.

8.4.3 Customizable Rules

YARA rules can be modified or extended to identify additional indicators. The project uses multiple custom rules covering suspicious PowerShell, command, scripting, and test indicators.

Benefit: Security analysts can adapt detection logic according to new investigation requirements.

8.4.4 Supports Threat Hunting

The scanner supports recursive scanning of a designated directory structure to search for predefined indicators of compromise (IOCs).

Benefit: Helps security analysts perform proactive searches for suspicious artifacts across multiple directories.

8.4.5 Endpoint Security Enhancement

An endpoint scanning component was implemented using PowerShell. It scans selected endpoint locations such as Downloads, Desktop, and Documents using the YARA rules.

Benefit: Provides an additional file-level detection layer that can complement existing endpoint security and antivirus solutions.

8.5 Use Cases

8.5.1 Malware Detection and Analysis

The scanner can be used during malware analysis to identify files containing predefined suspicious indicators.

8.5.2 Endpoint Security Enhancement

Selected endpoint directories can be scanned to identify files matching security detection rules.

8.5.3 Incident Response

When a suspicious file is detected, the incident-response component records the associated YARA rule, file path, detection time, and SHA-256 hash.

8.5.4 Threat Hunting Operations

Security analysts can recursively scan designated directories to search for known indicators of compromise.

8.5.5 Cybersecurity Training and Labs

The project provides a controlled environment for learning YARA rule creation, malware detection concepts, recursive scanning, threat hunting, and incident-response procedures.

8.6 Conclusion

The Simple Malware Scanner Using YARA provides a lightweight and customizable approach to file-level malware detection. By combining YARA's rule-based pattern matching with PowerShell automation, the project supports detection, recursive scanning, endpoint scanning, threat hunting, and basic incident-response evidence collection.

The implementation demonstrates how customized detection rules can be applied to controlled test samples to identify potentially suspicious files and generate useful security reports.

8.7 Outcome of the Project

Technical Outcomes

The project achieved the following technical outcomes:

- Successful implementation of YARA-based file scanning.
- Successful creation and testing of custom YARA detection rules.
- Successful detection of controlled suspicious test samples.
- Successful validation using a clean test file.
- Successful recursive scanning of nested directories.
- Successful automated scanning using PowerShell.
- Successful measurement of scan duration.
- Successful endpoint directory scanning.
- Successful threat-hunting implementation.
- Successful incident-response evidence collection.
- Successful SHA-256 hash generation for detected files.
- Successful generation of scan, endpoint, threat-hunting, and incident-response reports.

Final Test Result

The final scanner test produced the following result:

```
Total Files Scanned : 6
Rule Matches        : 5
Scan Duration       : 0.056 Seconds
```

Five controlled test files triggered YARA rules, while one clean test file produced no rule match.

Operational and Educational Impact

The project provides practical experience in:

- Malware detection concepts.
- YARA rule development.
- File-level security analysis.

- PowerShell security automation.
- Recursive file scanning.
- Endpoint security.
- Threat hunting.
- Incident-response procedures.
- Digital evidence identification through SHA-256 hashing.

The project demonstrates how a simple YARA-based scanner can serve as a useful educational and security-analysis tool for identifying potentially suspicious artifacts.

9. EXISTING SYSTEM

The existing approach to malware detection commonly relies on antivirus software, endpoint security platforms, manual file inspection, and signature-based detection mechanisms. These technologies play an important role in protecting systems, but they may not always provide the customization and flexibility required for specific malware analysis and threat-hunting activities.

9.1 Traditional Antivirus Detection

Traditional antivirus solutions generally use a combination of malware signatures, behavioral analysis, heuristics, and other detection technologies to identify malicious files. They are designed to provide continuous protection for endpoints and systems.

However, security analysts may have limited control over the exact file-level detection patterns used by commercial antivirus products.

9.2 Manual File Analysis

Another approach is manually inspecting suspicious files and directories. Analysts may examine file names, extensions, strings, scripts, and other characteristics to determine whether a file appears suspicious.

Manual analysis can become time-consuming when a large number of files or directories need to be investigated.

9.3 Signature-Based Detection

Signature-based detection identifies files by comparing them against known signatures or patterns. This approach can be effective for known threats but may have limitations when dealing with new, customized, or previously unknown artifacts.

9.4 Limitations of the Existing System

The major limitations identified are:

- Limited customization of vendor-controlled detection mechanisms.
- Difficulty in rapidly creating organization-specific detection patterns.
- Manual investigation can be time-consuming.
- Searching large directory structures manually is inefficient.
- Limited visibility into which specific custom indicator triggered a detection.
- Additional tools may be required for specialized threat-hunting activities.
- Incident-response evidence may need to be collected separately.

9.5 Need for an Improved Approach

There is a need for a lightweight and customizable detection mechanism that allows security analysts to define their own detection logic and apply it to selected files or directories.

YARA addresses this requirement by allowing users to create rules based on textual strings, binary patterns, and logical conditions. When combined with PowerShell automation, YARA can be used to create a simple scanning workflow capable of recursive scanning, endpoint scanning, threat hunting, and report generation.

Therefore, the proposed **Simple Malware Scanner Using YARA** is designed to provide an additional and customizable file-level detection capability that can complement existing security solutions.

10. PROPOSED SYSTEM

The proposed system is a **Simple Malware Scanner Using YARA** designed to identify potentially suspicious files by comparing them against customized YARA detection rules. The system provides a lightweight and flexible approach to file-level security analysis and can be used for malware detection, endpoint security enhancement, threat hunting, and incident-response activities.

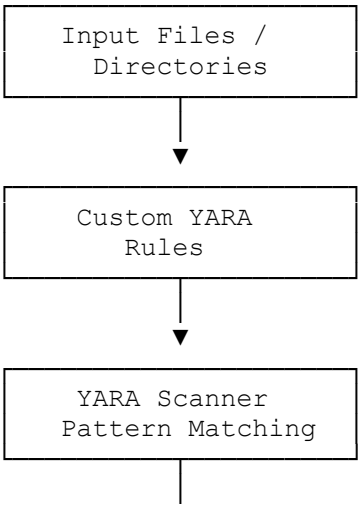
The solution combines the **YARA scanning engine** with **PowerShell automation**. YARA performs the actual pattern matching, while PowerShell scripts automate scanning operations, process the results, and generate security reports.

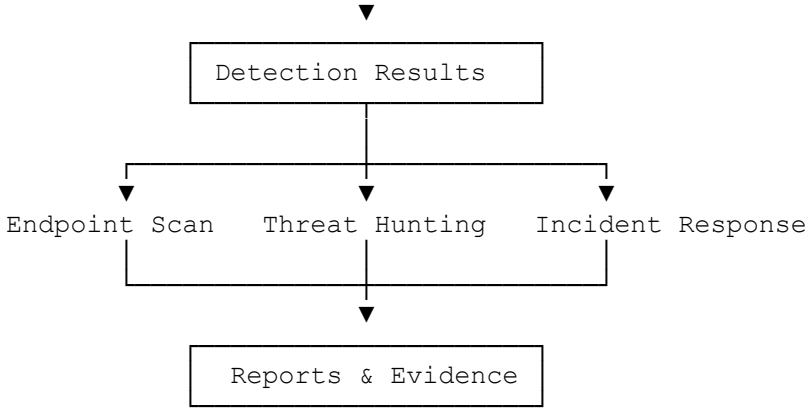
10.1 System Overview

The proposed system consists of the following major components:

1. **YARA Detection Engine**
2. **Custom YARA Rules**
3. **Controlled Test Samples**
4. **Main Malware Scanner**
5. **Recursive Scanning**
6. **Endpoint Scanner**
7. **Threat Hunting Module**
8. **Incident Response Module**
9. **SHA-256 Evidence Collection**
10. **Security Reports**

The overall workflow is:





10.2 YARA Detection Engine

YARA is responsible for comparing scanned files against the defined detection rules. Each rule contains strings or patterns and logical conditions that determine whether a file matches.

When a match occurs, YARA returns the corresponding rule name and the path of the detected file.

10.3 Custom YARA Rules

The project uses custom rules to detect controlled suspicious indicators. The rules include indicators associated with:

- Suspicious PowerShell activity.
- Suspicious command execution.
- Suspicious scripting activity.
- Controlled malware-detection test patterns.

The rules can be modified or extended when additional indicators need to be detected.

10.4 Main Malware Scanner

The main scanner is implemented using PowerShell and executes YARA against the project sample directory.

The scanner provides:

- Scan start time.
- Target directory.
- Total files scanned.
- Number of rule matches.
- Scan duration.
- Detection results.

- Output report.

10.5 Recursive Scanning

The scanner supports recursive scanning using YARA's recursive scanning capability. This allows files located inside nested directories to be examined automatically.

For example:

```
samples
└── nested
    └── level2
        └── deep_test.txt
```

The scanner successfully detected the controlled test file located in the nested directory.

10.6 Endpoint Security Scanning

The endpoint scanning component examines selected directories such as:

- Downloads
- Desktop
- Documents

The purpose is to demonstrate how YARA can provide an additional file-level detection capability on an endpoint.

10.7 Threat Hunting

The threat-hunting component recursively scans a designated directory structure containing simulated endpoint locations such as Documents, Downloads, Temp, and USB.

This demonstrates how analysts can search multiple locations for predefined suspicious indicators.

10.8 Incident Response

The incident-response component assists in documenting detected artifacts. Information associated with a detection can be recorded for further investigation.

The collected information includes:

- YARA detection rule.
- File path.
- Detection timestamp.
- SHA-256 hash.

This information provides basic evidence that can be used during security investigations.

10.9 Reporting

The proposed system generates separate reports for different security activities, including:

```
results
├── scan_results.txt
├── endpoint_report.txt
├── incident_report.txt
└── threat_hunting_report.txt
```

These reports provide a record of the scanning and detection activities performed during the project.

10.10 Advantages of the Proposed System

The proposed system provides the following advantages:

- Customizable detection rules.
- Lightweight scanning.
- Rapid file analysis.
- Recursive directory scanning.
- Automated scanning through PowerShell.
- Endpoint directory scanning.
- Threat-hunting capability.
- Incident-response evidence collection.
- SHA-256 hash identification.
- Automated report generation.
- Suitable for cybersecurity training and controlled laboratory environments.

The proposed system therefore provides a practical demonstration of how YARA can be integrated into a simple security-scanning workflow for malware detection and investigation.

11. REQUIREMENTS SPECIFICATION

The requirements specification defines the hardware, software, and environmental requirements necessary to implement and operate the Simple Malware Scanner Using YARA.

The project was implemented and tested in a Windows environment using the YARA command-line engine and PowerShell automation.

11.1 Hardware Requirements

The scanner does not require specialized hardware. A standard personal computer or laptop is sufficient for implementing and testing the project.

Component	Minimum Requirement	Recommended
Processor	Dual-Core CPU	Intel Core i3 / equivalent or higher
RAM	4 GB	8 GB or higher
Storage	2 GB free space	10 GB or more
Display	Standard monitor	Any modern display
Keyboard/Mouse	Required	Required
USB Storage	Optional	For removable-media scanning demonstration
Network	Not required for local scanning	Internet connection for downloading tools/updates

The hardware requirements are relatively low because YARA is a lightweight pattern-matching engine and the project performs targeted file scanning rather than continuous enterprise-scale endpoint monitoring.

11.2 Software Requirements

The following software components are required for the implementation:

Software	Purpose
Windows 10/11	Operating environment
YARA	Malware pattern matching and detection
PowerShell	Scanner automation and report generation
Command Prompt	Command-line execution and testing
Text Editor	Creating and modifying YARA rules and test files

11.2.1 YARA

YARA is the primary detection engine used by the project. It processes the custom `.yar` rule files and scans the specified files or directories for matching patterns.

The project uses the Windows YARA executable:

```
yara64.exe
```

11.2.2 PowerShell

PowerShell is used to automate the scanning workflow. The project contains separate scripts for:

```
scanner.ps1
endpoint_scan.ps1
incident_response.ps1
threat_hunt.ps1
```

These scripts automate different security activities and generate corresponding reports.

11.2.3 YARA Rule Files

The project uses the following rule files:

```
malware_rules.yar
master_rules.yar
test_malware.yar
```

These files contain the detection logic used during scanning.

11.2.4 Test Environment

Controlled test samples were created specifically to validate the scanner. These samples contain predefined indicators designed to trigger the corresponding YARA rules.

The project also includes a clean sample to verify that a file without matching indicators does not generate a detection.

11.2.5 Project Storage Structure

The project is organized into separate directories for rules, samples, threat-hunting data, scripts, and generated reports.

```
YARA-Project
│
├── rules
├── samples
├── ThreatHunt
├── results
│
├── scanner.ps1
├── endpoint_scan.ps1
├── incident_response.ps1
└── threat_hunt.ps1
```

This organization makes the project easier to maintain, test, and demonstrate.

11.3 Functional Requirements

The system should be capable of performing the following functions:

1. Load and process YARA rules.

2. Scan individual files.
3. Scan directories recursively.
4. Identify files matching defined rules.
5. Display detection results.
6. Count rule matches.
7. Count scanned files.
8. Measure scan duration.
9. Save scan results to a report.
10. Scan selected endpoint directories.
11. Perform threat-hunting scans.
12. Generate incident-response information.
13. Calculate SHA-256 hashes for relevant files.
14. Store generated reports for later analysis.

11.4 Non-Functional Requirements

Performance

The scanner should perform file analysis efficiently and provide scan-duration information.

Usability

The system should be executable through simple PowerShell or command-line commands.

Maintainability

YARA rules should be easy to modify, add, or remove without redesigning the entire scanner.

Reliability

The scanner should consistently produce detection results for files containing the indicators defined by the corresponding rules.

Security

Testing should be performed using controlled samples in an isolated or authorized environment rather than relying on live malicious files.

Scalability

The recursive scanning capability allows the scanner to be applied to larger directory structures without manually specifying every individual file.

11.5 Requirements Summary

The Simple Malware Scanner Using YARA requires only a standard Windows computer, YARA, and PowerShell. The lightweight requirements make the project suitable for cybersecurity education, laboratory environments, malware-analysis exercises, threat hunting demonstrations, and basic endpoint file scanning.

12. TECHNOLOGY OVERVIEW

The Simple Malware Scanner Using YARA uses a combination of security and automation technologies to perform rule-based file detection, automated scanning, threat hunting, and incident-response evidence collection.

The primary technologies used in this project are **YARA**, **PowerShell**, and **SHA-256**.

12.1 YARA

YARA is a rule-based pattern-matching tool designed to help security researchers and analysts identify and classify files based on specific characteristics.

YARA rules consist of two major components:

- **Strings or patterns** that describe characteristics to search for.
- **Conditions** that determine when a rule should generate a match.

A simplified YARA rule can be represented as:

```
rule Example_Detection
{
    strings:
        $indicator = "suspicious_pattern"

    condition:
        $indicator
}
```

When YARA scans a file and the condition evaluates to true, the rule name and file path are returned as a detection result.

Role of YARA in the Project

YARA is the core detection engine of the project. It is responsible for:

- Loading custom detection rules.
- Scanning files.
- Matching predefined indicators.
- Detecting suspicious patterns.
- Supporting recursive directory scanning.
- Reporting matched rule names and file paths.

The Windows executable used in the project is:

```
yara64.exe
```

The project includes multiple YARA rule files containing customized detection logic.

12.2 PowerShell

PowerShell is a command-line shell and scripting environment available on Windows systems. It provides capabilities for executing commands, automating tasks, processing files, and generating reports.

In this project, PowerShell is used as the automation layer around the YARA engine.

Role of PowerShell in the Project

PowerShell scripts are used to:

- Execute YARA scans.
- Specify rule and target paths.
- Perform automated scanning.
- Process YARA output.
- Count rule matches.
- Measure scan duration.
- Generate scan reports.
- Perform endpoint scanning.
- Support threat-hunting activities.
- Collect incident-response information.

The project contains four primary PowerShell scripts:

```
scanner.ps1
endpoint_scan.ps1
incident_response.ps1
threat_hunt.ps1
```

The use of PowerShell allows multiple YARA operations to be combined into a practical automated scanning workflow.

12.3 SHA-256

SHA-256 (Secure Hash Algorithm 256-bit) is a cryptographic hash function that generates a fixed-length 256-bit hash value from input data.

A SHA-256 hash can be represented as a hexadecimal value such as:

```
A1B2C3D4E5F6...
```

The actual hash depends on the contents of the file.

Role of SHA-256 in the Project

SHA-256 is used as part of the incident-response evidence collection process.

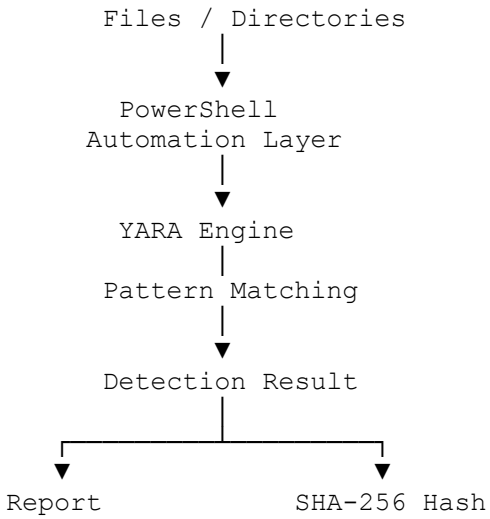
When a suspicious file is identified, its SHA-256 hash can be recorded along with other information such as:

- File path.
- YARA rule that triggered.
- Detection timestamp.
- File hash.

This provides a unique content identifier that can assist with evidence tracking and further analysis.

12.4 Relationship Between the Technologies

The three technologies work together as follows:



YARA performs the detection, PowerShell automates the scanning and reporting workflow, and SHA-256 provides file-content identification for incident-response evidence.

12.5 Advantages of the Technology Stack

The selected technology stack provides several benefits:

- **YARA:** Flexible and customizable rule-based detection.
- **PowerShell:** Native Windows automation and scripting capabilities.
- **SHA-256:** Reliable file-content identification.
- **Combined workflow:** Provides detection, automation, reporting, threat hunting, and basic incident-response support.

The technologies are lightweight and suitable for a controlled cybersecurity laboratory project demonstrating practical malware detection concepts.

13. SYSTEM ARCHITECTURE

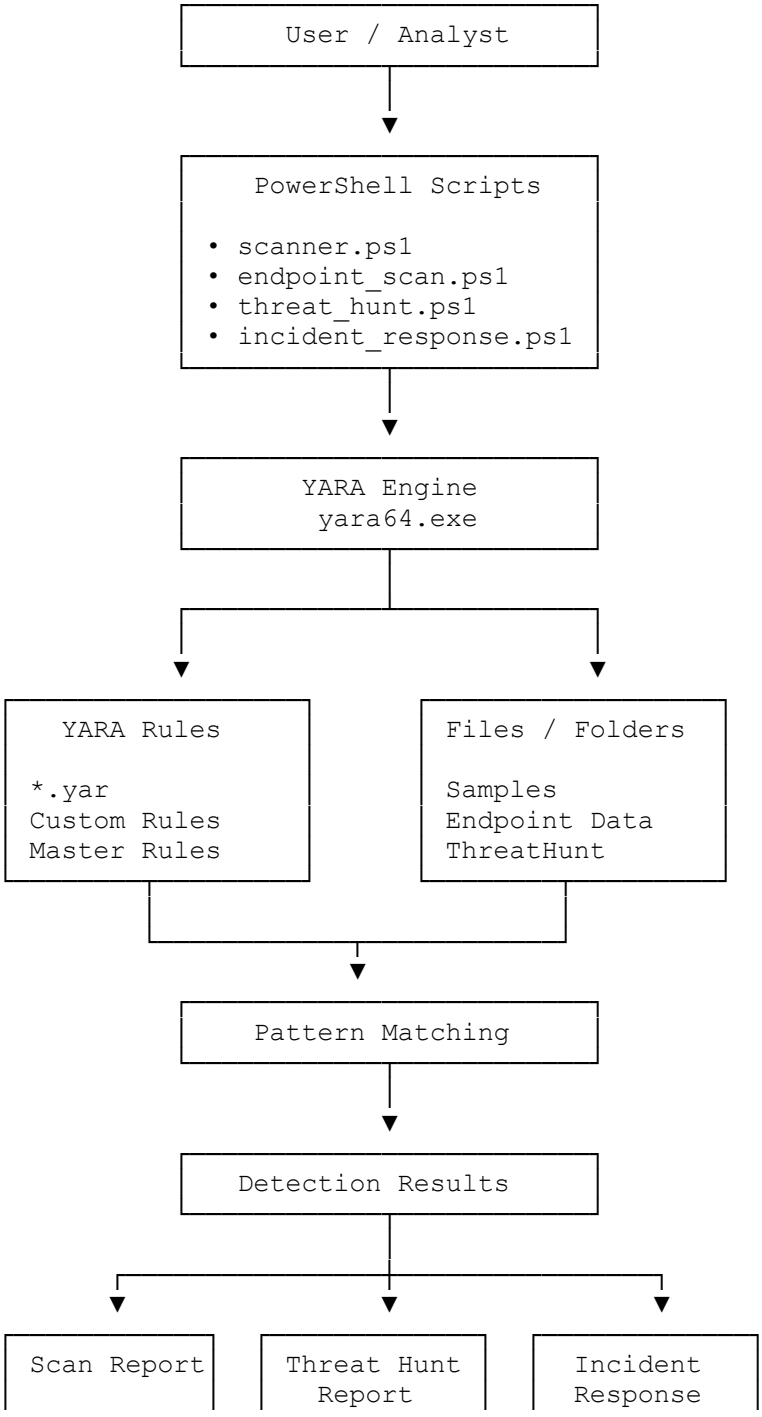
The architecture of the Simple Malware Scanner Using YARA is designed around a rule-based detection engine combined with PowerShell automation. The system accepts files or

directories as input, applies customized YARA rules, processes the resulting detections, and stores the results in security reports.

The architecture is divided into several logical components to make the system easy to understand, maintain, and extend.

13.1 High-Level Architecture

The high-level architecture of the project is:





13.2 Input Layer

The input layer contains the files and directories that need to be analyzed.

The project uses several input locations:

```
C:\Users\ubaid\YARA-Project\samples
C:\Users\ubaid\YARA-Project\ThreatHunt
```

The `samples` directory contains controlled test files used to validate the YARA detection rules.

The `ThreatHunt` directory contains simulated endpoint locations such as:

```
ThreatHunt
├── Documents
├── Downloads
├── Temp
└── USB
```

13.3 Rule Layer

The rule layer contains the YARA detection logic.

The project includes:

```
rules
├── malware_rules.yar
├── master_rules.yar
└── test_malware.yar
```

The rules define the patterns and conditions used to identify controlled suspicious indicators.

The rule layer is separated from the scanner scripts so that detection logic can be updated without changing the complete scanning workflow.

13.4 Detection Engine

The YARA executable acts as the core detection engine.

```
yara64.exe
```

It receives the selected rule file and target file or directory and performs pattern matching.

For recursive scanning, the `-r` option allows YARA to examine files inside nested directories.

A simplified operation is:

```
YARA Rules + Target Files
          ↓
     Pattern Matching
          ↓
    Rule Evaluation
          ↓
 Detection / No Detection
```

13.5 Automation Layer

PowerShell provides the automation layer around the YARA engine.

The project contains four main scripts:

```
scanner.ps1
endpoint_scan.ps1
threat_hunt.ps1
incident_response.ps1
```

Each script performs a specific security function.

Main Scanner

`scanner.ps1` performs the primary malware scanning operation.

Endpoint Scanner

`endpoint_scan.ps1` scans selected endpoint directories.

Threat Hunting

`threat_hunt.ps1` recursively searches the designated threat-hunting directory.

Incident Response

`incident_response.ps1` collects detection information and supporting evidence such as SHA-256 hashes.

13.6 Processing Layer

The processing layer receives the output generated by YARA and processes the information.

The main scanner records:

- Scan start time.
- Target directory.
- Total files scanned.
- Rule matches.
- Scan duration.

- Detection results.

PowerShell processes these results and stores them in the appropriate report file.

13.7 Evidence and Reporting Layer

The final layer stores the results generated by the security activities.

The project uses:

```
results
├── scan_results.txt
├── endpoint_report.txt
├── incident_report.txt
└── threat_hunting_report.txt
```

These reports provide records of the scanning and detection activities.

For incident-response purposes, SHA-256 hashes provide an additional identifier for relevant files.

13.8 Data Flow

The complete data flow can be summarized as:

```
User selects target
        ↓
PowerShell script starts
        ↓
YARA rules loaded
        ↓
Files/directories scanned
        ↓
YARA performs pattern matching
        ↓
Rule matches identified
        ↓
PowerShell processes results
        ↓
Detection information recorded
        ↓
Reports generated
        ↓
Incident-response evidence collected when required
```

13.9 Architecture Benefits

The architecture provides several benefits:

- Separation of detection rules and scanning logic.
- Easy modification of YARA rules.
- Automated scanning through PowerShell.

- Support for recursive directory scanning.
- Separate workflows for endpoint security, threat hunting, and incident response.
- Centralized storage of generated reports.
- Simple and understandable project structure.
- Suitable for controlled cybersecurity laboratory environments.

The architecture therefore provides a structured foundation for implementing and demonstrating the Simple Malware Scanner Using YARA.

14. PROJECT DIRECTORY STRUCTURE

The project is organized into separate directories for YARA rules, controlled test samples, threat-hunting data, security scripts, and generated reports. This modular structure makes the project easier to understand, maintain, test, and demonstrate.

The final project directory structure is shown below:

```
YARA-Project
|
```

```
├── endpoint_scan.ps1
├── incident_response.ps1
├── scanner.ps1
├── threat_hunt.ps1
│
├── results
│   ├── endpoint_report.txt
│   ├── incident_report.txt
│   ├── scan_results.txt
│   └── threat_hunting_report.txt
│
├── rules
│   ├── malware_rules.yar
│   ├── master_rules.yar
│   └── test_malware.yar
│
├── samples
│   ├── clean_sample.txt
│   ├── command_test.txt
│   ├── powershell_test.txt
│   ├── script_test.txt
│   ├── test_sample.txt
│   │
│   └── nested
│       └── level2
│           └── deep_test.txt
│
└── ThreatHunt
    ├── Documents
    │   ├── clean_sample.txt
    │   ├── notes.txt
    │   ├── script_test.txt
    │   └── startup_script.txt
    │
    ├── Downloads
    │   ├── invoice.txt
    │   └── powershell_test.txt
    │
    ├── Temp
    │   └── service.txt
    │
    └── USB
        └── command_test.txt
```

14.1 Root Project Directory

The root directory is:

```
C:\Users\ubaid\YARA-Project
```

It contains the PowerShell automation scripts and the main project directories.

14.2 Rules Directory

The `rules` directory contains the YARA rule files used by the scanner.

```
rules
```

```
├── malware_rules.yar
├── master_rules.yar
└── test_malware.yar
```

malware_rules.yar

Contains the customized detection rules used to identify predefined suspicious indicators.

master_rules.yar

Acts as the primary rule file used by the automated scanner and combines the required detection logic for the project.

test_malware.yar

Contains the test detection rule used during the initial validation of the YARA installation and scanning functionality.

14.3 Samples Directory

The `samples` directory contains controlled files used to test the detection capability.

```
samples
├── clean_sample.txt
├── command_test.txt
├── powershell_test.txt
├── script_test.txt
├── test_sample.txt
└── nested
    └── level2
        └── deep_test.txt
```

The directory contains both matching and non-matching files.

The nested directory is specifically used to demonstrate recursive scanning.

For example:

```
samples
└── nested
    └── level2
        └── deep_test.txt
```

The scanner successfully detected the test file located at this nested level.

14.4 ThreatHunt Directory

The `ThreatHunt` directory simulates different locations that could be investigated during a threat-hunting exercise.

```
ThreatHunt
├── Documents
├── Downloads
├── Temp
└── USB
```

These directories contain controlled test artifacts representing different storage locations.

This structure allows the threat-hunting script to demonstrate recursive searching across multiple locations.

14.5 Results Directory

The `results` directory stores the reports generated by the project.

```
results
├── endpoint_report.txt
├── incident_report.txt
├── scan_results.txt
└── threat_hunting_report.txt
```

scan_results.txt

Stores the results produced by the main malware scanner.

endpoint_report.txt

Stores the results from endpoint security scanning.

incident_report.txt

Stores information collected during incident-response analysis.

threat_hunting_report.txt

Stores the results generated during the threat-hunting process.

14.6 PowerShell Scripts

Four PowerShell scripts are stored in the root project directory.

scanner.ps1

The main malware-scanning script.

Its functions include:

- Starting the YARA scan.
- Scanning the sample directory.
- Counting files.
- Counting rule matches.
- Measuring scan duration.
- Displaying detection results.
- Saving results to a report.

endpoint_scan.ps1

Performs YARA-based scanning against selected endpoint directories such as Downloads, Desktop, and Documents.

threat_hunt.ps1

Performs recursive threat hunting against the designated `ThreatHunt` directory.

incident_response.ps1

Supports incident-response evidence collection by documenting detected files and calculating SHA-256 hashes.

14.7 Project Organization Benefits

The directory structure provides several benefits:

- Separates detection rules from scripts.
- Separates test samples from project output.
- Keeps generated reports organized.
- Makes recursive scanning easy to demonstrate.
- Provides dedicated locations for threat-hunting exercises.
- Makes the project easier to maintain and present.
- Allows each component to be tested independently.

The organized directory structure therefore provides a clear foundation for the implementation, testing, and documentation of the Simple Malware Scanner Using YARA.

15. YARA RULE DESIGN AND DEVELOPMENT

YARA rules form the core detection mechanism of the Simple Malware Scanner. The rules define the characteristics that the scanner searches for within files. When the conditions specified by a rule are satisfied, YARA reports the corresponding rule name and the file in which the match occurred.

The project uses custom rules designed specifically for a controlled cybersecurity testing environment. The rules are intended to demonstrate how suspicious indicators can be identified through rule-based pattern matching.

15.1 Rule-Based Detection

YARA uses a rule structure consisting primarily of:

- Rule name.
- Strings or patterns.
- Conditions.

A simplified representation of a YARA rule is:

```
rule Example_Rule
{
    strings:
        $indicator = "example_pattern"

    condition:
        $indicator
}
```

The `strings` section defines the indicators that YARA searches for, while the `condition` section determines when the rule should generate a match.

The general detection process is:

```
File
  ↓
YARA reads file content
  ↓
Searches for defined indicators
  ↓
Evaluates rule condition
  ↓
Condition satisfied?
  ├── YES → Rule Match
  └── NO  → No Match
```

15.2 Custom YARA Rules

The project includes customized rules designed to identify controlled suspicious indicators.

The main detection categories include:

15.2.1 Suspicious PowerShell Indicator

The `Suspicious_PowerShell_Indicator` rule is designed to identify predefined PowerShell-related indicators in the controlled test environment.

During testing, the rule successfully detected:

`powershell_test.txt`

The scanner reported:

`Suspicious_PowerShell_Indicator`

15.2.2 Suspicious Command Indicator

The `Suspicious_Command_Indicator` rule is designed to identify predefined command-execution indicators.

During testing, the rule successfully detected:

`command_test.txt`

The scanner reported:

`Suspicious_Command_Indicator`

15.2.3 Suspicious Script Indicator

The `Suspicious_Script_Indicator` rule is designed to identify predefined scripting-related indicators.

During testing, the rule successfully detected:

`script_test.txt`

The scanner reported:

`Suspicious_Script_Indicator`

15.2.4 Test Malware Detection Rule

The `Test_Malware_Detection` rule was created to validate the basic operation of the YARA scanner.

It successfully detected:

`test_sample.txt`

It also successfully detected the same type of controlled test indicator in a nested directory:

`samples\nested\level2\deep_test.txt`

This provided evidence that the scanner could detect matching files at different directory levels.

15.3 Clean File Validation

A clean test file was included to verify that the scanner does not report every file as suspicious.

The file:

```
clean_sample.txt
```

was scanned against the YARA rules and produced no rule match.

This provides a basic negative test for the detection system.

The testing workflow was therefore:

```
Suspicious Test Samples
        ↓
   Rule Matches
        ↓
    Detection

Clean Test Sample
        ↓
   No Rule Match
        ↓
    No Detection
```

15.4 Master Rule File

The project contains a master YARA rule file:

```
master_rules.yar
```

The master rule file is used as the primary rule set by the automated scanner.

It provides a centralized location for the detection rules required by the project and simplifies scanner execution.

The main scanner executes the rule set against the target directory rather than requiring each rule to be executed individually.

15.5 Rule Customization

One of the main advantages of YARA is the ability to modify detection rules according to investigation requirements.

A security analyst can:

1. Identify a suspicious indicator.
2. Add the indicator to a YARA rule.
3. Define an appropriate condition.
4. Test the rule against controlled samples.
5. Add the rule to the operational rule set.
6. Scan the required files or directories.

This makes the detection system adaptable to new investigation requirements.

15.6 Rule Testing

The rules were tested individually and collectively during project implementation.

The final scanner test produced:

```
Total Files Scanned : 6
Rule Matches        : 5
Scan Duration       : 0.056 Seconds
```

The detected rules included:

```
Suspicious_PowerShell_Indicator
Suspicious_Command_Indicator
Suspicious_Script_Indicator
Test_Malware_Detection
```

The recursive test also confirmed detection of:

```
nested\level2\deep_test.txt
```

15.7 Importance of YARA Rule Development

Effective YARA rules are important because the quality of detection depends on the indicators and conditions defined by the analyst.

Poorly designed rules may generate unnecessary matches, while overly specific rules may fail to identify relevant files.

Therefore, rule development should consider:

- Relevant indicators.
- Appropriate conditions.
- Test coverage.
- False-positive possibilities.
- Maintainability.
- Regular rule updates.

The custom rules developed in this project demonstrate the fundamental process of creating, testing, and applying YARA-based detection logic in a controlled environment.

16. SYSTEM IMPLEMENTATION

The Simple Malware Scanner Using YARA was implemented in a Windows environment using the YARA command-line engine and PowerShell. The implementation was performed in a controlled project environment using predefined test artifacts.

The implementation consists of YARA installation and validation, rule creation, sample preparation, individual scanning, recursive scanning, automated scanning, endpoint scanning, threat hunting, and incident-response evidence collection.

16.1 YARA Installation

YARA was installed on the Windows system using the Windows Package Manager (`winget`).

The available YARA package was verified using:

```
winget search yara
```

The package information showed the VirusTotal YARA package.

The installation/package status was verified using:

```
winget list --id VirusTotal.YARA -e
```

The YARA package was identified as installed.

Because the portable YARA package was extracted manually, the executable was placed at:

```
C:\Users\ubaid\YARA\yara64.exe
```

The installation was then validated by executing:

```
yara64.exe
```

YARA displayed its command-line usage information, confirming that the executable was functioning correctly.

16.2 YARA Version Validation

The installed YARA executable was tested to confirm that it could execute successfully.

The executable used in the project was:

```
C:\Users\ubaid\YARA\yara64.exe
```

The successful display of the YARA usage information confirmed that the executable could be launched from the project environment.

16.3 Project Directory Creation

A dedicated project directory was created:

```
C:\Users\ubaid\YARA-Project
```

The project was organized into separate directories:

```
YARA-Project
├── results
├── rules
├── samples
└── ThreatHunt
```

PowerShell scripts were stored in the project root directory.

16.4 YARA Rule Creation

Custom YARA rules were created and stored in the `rules` directory.

The rule files include:

```
malware_rules.yar
master_rules.yar
test_malware.yar
```

The rules were designed to identify controlled suspicious indicators associated with PowerShell, command execution, scripting activity, and test detection patterns.

16.5 Controlled Test Sample Creation

Controlled test artifacts were created in the `samples` directory to validate the detection rules.

The test directory contains:

```
samples
├── clean_sample.txt
├── command_test.txt
├── powershell_test.txt
├── script_test.txt
├── test_sample.txt
└── nested
    └── level2
        └── deep_test.txt
```

The files were created for controlled testing and are not presented as real malware samples.

16.6 Individual File Scanning

The initial YARA functionality was tested by scanning an individual controlled sample.

The following command was used:

```
yara64.exe C:\Users\ubaid\YARA-Project\rules\test_malware.yar
C:\Users\ubaid\YARA-Project\samples\test_sample.txt
```

The scanner returned:

```
Test_Malware_Detection C:\Users\ubaid\YARA-Project\samples\test_sample.txt
```

This confirmed that the YARA rule successfully matched the controlled test artifact.

16.7 Clean File Testing

A clean test file was scanned to perform negative validation.

The file used was:

```
C:\Users\ubaid\YARA-Project\samples\clean_sample.txt
```

No YARA rule match was returned.

This demonstrated that the scanner could distinguish between a controlled file containing the test indicator and a clean test file without a matching indicator.

16.8 Detection Rule Testing

The customized detection rules were tested against the corresponding controlled samples.

For example, the PowerShell-related rule successfully detected:

```
Suspicious_PowerShell_Indicator
```

The command-related rule successfully detected:

```
Suspicious_Command_Indicator
```

The scripting-related rule successfully detected:

```
Suspicious_Script_Indicator
```

The test detection rule successfully detected:

```
Test_Malware_Detection
```

16.9 Recursive Scanning Implementation

Recursive scanning was implemented using the YARA `-r` option.

The following command was used:

```
yara64.exe -r C:\Users\ubaid\YARA-Project\rules\malware_rules.yar
C:\Users\ubaid\YARA-Project\samples
```

The scan successfully detected files in both the main directory and nested directories.

A significant result was the detection of:

```
C:\Users\ubaid\YARA-Project\samples\nested\level2\deep_test.txt
```

This demonstrated that the scanner could traverse nested directories automatically.

16.10 Automated Scanner Implementation

A PowerShell script named:

```
scanner.ps1
```

was developed to automate the primary scanning process.

The script performs the following operations:

1. Defines the YARA executable path.
2. Defines the YARA rule path.

3. Defines the target sample directory.
4. Starts the YARA scan.
5. Processes the returned results.
6. Counts scanned files.
7. Counts rule matches.
8. Calculates scan duration.
9. Displays detection results.
10. Saves the results to a report.

The resulting report is stored at:

```
C:\Users\ubaid\YARA-Project\results\scan_results.txt
```

16.11 Final Automated Scan

The automated scanner was executed using:

```
.\scanner.ps1
```

The final scan produced:

```
=======================================
 SIMPLE MALWARE SCANNER USING YARA
=======================================

Scan Started : 31-08-2026 12:47:43
Target Folder: C:\Users\ubaid\YARA-Project\samples

========== SCAN SUMMARY ==========
Total Files Scanned : 6
Rule Matches        : 5
Scan Duration       : 0.056 Seconds
==================================
```

The detection results were:

```
Suspicious_PowerShell_Indicator
Suspicious_Command_Indicator
Test_Malware_Detection
Test_Malware_Detection
Suspicious_Script_Indicator
```

The nested test file was also detected successfully.

16.12 Endpoint Scanning Implementation

An endpoint scanning script named:

```
endpoint_scan.ps1
```

was implemented to demonstrate scanning of selected endpoint locations.

The script was executed using:

```
.\endpoint_scan.ps1
```

The scan processed locations including:

```
Downloads
Desktop
Documents
```

The execution reported suspicious indicators in the tested directories.

The resulting report was saved to:

```
C:\Users\ubaid\YARA-Project\results\endpoint_report.txt
```

16.13 Threat Hunting Implementation

A dedicated threat-hunting directory was created:

```
C:\Users\ubaid\YARA-Project\ThreatHunt
```

It contains simulated locations:

```
ThreatHunt
├── Documents
├── Downloads
├── Temp
└── USB
```

The threat-hunting PowerShell script:

```
threat_hunt.ps1
```

was used to scan these locations for predefined indicators.

The generated report was stored as:

```
C:\Users\ubaid\YARA-Project\results\threat_hunting_report.txt
```

16.14 Incident Response Implementation

The project also includes:

```
incident_response.ps1
```

This component was designed to support basic incident-response evidence collection.

When a relevant file is identified, information such as the file path, detection rule, timestamp, and SHA-256 hash can be recorded.

The resulting report is stored at:

```
C:\Users\ubaid\YARA-Project\results\incident_report.txt
```

16.15 SHA-256 Evidence Collection

SHA-256 hashing was incorporated into the incident-response workflow.

The hash provides a content-based identifier for a file and can be recorded as part of the investigation evidence.

The workflow is:

```
Detected File
     ↓
Calculate SHA-256
     ↓
Record Hash
     ↓
Associate with Detection
     ↓
Store in Incident Report
```

This allows the analyst to maintain a consistent identifier for a file during further investigation.

16.16 Report Generation

The implementation generates separate reports for the different security functions:

```
results
├── scan_results.txt
├── endpoint_report.txt
├── incident_report.txt
└── threat_hunting_report.txt
```

Separating the reports makes it easier to review the results of each security activity independently.

16.17 Implementation Summary

The complete implementation successfully combines YARA and PowerShell into a basic malware-detection workflow.

The implemented system supports:

- YARA rule-based detection.
- Controlled sample testing.
- Clean-file validation.
- Recursive scanning.
- Automated scanning.

- Scan-performance measurement.
- Endpoint scanning.
- Threat hunting.
- Incident-response evidence collection.
- SHA-256 file identification.
- Automated report generation.

The implementation provides the technical foundation for the testing and results discussed in the following sections.

17. TESTING METHODOLOGY

Testing was performed to verify the functionality, reliability, and expected behavior of the Simple Malware Scanner Using YARA. The testing process used controlled test artifacts and predefined YARA rules rather than live malware.

The testing methodology covered individual file detection, clean-file validation, recursive scanning, automated scanning, endpoint scanning, threat hunting, and incident-response functionality.

17.1 Testing Objectives

The primary objectives of testing were:

- Verify that YARA was installed and functioning correctly.
- Verify that custom YARA rules could detect their intended test indicators.
- Verify that clean files did not generate unintended matches.
- Verify recursive scanning functionality.
- Verify automated PowerShell scanning.

- Verify scan statistics and timing.
- Verify endpoint scanning.
- Verify threat-hunting functionality.
- Verify incident-response evidence collection.
- Verify report generation.

17.2 Test Environment

Testing was performed in the following environment:

Component	Test Environment
Operating System	Windows
Detection Engine	YARA
YARA Executable	`yara64.exe`
Automation	PowerShell
Rule Format	`.yar`
Test Artifact Format	`.txt`
Hashing	SHA-256
Testing Type	Controlled laboratory testing

The test environment was organized under:

```
C:\Users\ubaid\YARA-Project
```

17.3 Test Data

The following controlled test files were used:

Test File	Purpose
`test_sample.txt`	Basic YARA detection test
`clean_sample.txt`	Negative/clean-file test
`powershell_test.txt`	PowerShell indicator detection
`command_test.txt`	Command indicator detection
`script_test.txt`	Script indicator detection
`deep_test.txt`	Recursive scanning test

The test artifacts were created for the project and should not be interpreted as actual malware.

17.4 Test Case 1 — YARA Installation Validation

Objective: Verify that the YARA executable can be executed.

Test Action:

```
yara64.exe
```

Expected Result:

YARA should display its command-line usage information.

Actual Result:

The YARA usage information was displayed successfully.

Status: PASS

17.5 Test Case 2 — Basic Rule Detection

Objective: Verify that a YARA rule detects its intended test indicator.

Test Command:

```
yara64.exe C:\Users\ubaid\YARA-Project\rules\test_malware.yar
C:\Users\ubaid\YARA-Project\samples\test_sample.txt
```

Expected Result:

The corresponding YARA rule should be reported.

Actual Result:

```
Test_Malware_Detection
```

Status: PASS

17.6 Test Case 3 — Clean File Validation

Objective: Verify that a clean test file does not produce a rule match.

Test File:

```
clean_sample.txt
```

Expected Result:

No YARA rule should match.

Actual Result:

No detection was reported.

Status: PASS

This test provides basic negative validation of the detection logic.

17.7 Test Case 4 — PowerShell Indicator Detection

Objective: Verify detection of the controlled PowerShell indicator.

Test File:

```
powershell_test.txt
```

Expected Result:

The `Suspicious_PowerShell_Indicator` rule should match.

Actual Result:

```
Suspicious_PowerShell_Indicator
```

Status: PASS

17.8 Test Case 5 — Command Indicator Detection

Objective: Verify detection of the controlled command indicator.

Test File:

```
command_test.txt
```

Expected Result:

The `Suspicious_Command_Indicator` rule should match.

Actual Result:

```
Suspicious_Command_Indicator
```

Status: PASS

17.9 Test Case 6 — Script Indicator Detection

Objective: Verify detection of the controlled scripting indicator.

Test File:

```
script_test.txt
```

Expected Result:

The `Suspicious_Script_Indicator` rule should match.

Actual Result:

```
Suspicious_Script_Indicator
```

Status: PASS

17.10 Test Case 7 — Recursive Scanning

Objective: Verify that YARA can scan files located inside nested directories.

The test file was located at:

```
samples\nested\level2\deep_test.txt
```

The recursive scan was performed using:

```
yara64.exe -r C:\Users\ubaid\YARA-Project\rules\malware_rules.yar
C:\Users\ubaid\YARA-Project\samples
```

Expected Result:

YARA should search the main directory and its nested subdirectories.

Actual Result:

```
Test_Malware_Detection C:\Users\ubaid\YARA-
Project\samples\nested\level2\deep_test.txt
```

Status: PASS

17.11 Test Case 8 — Automated Scanner

Objective: Verify the PowerShell-based automated scanner.

Test Command:

```
.\scanner.ps1
```

Expected Result:

The script should scan the sample directory and display scan statistics and detection results.

Actual Result:

```
Total Files Scanned : 6
Rule Matches        : 5
Scan Duration       : 0.056 Seconds
```

The expected detection results were displayed.

Status: PASS

17.12 Test Case 9 — Endpoint Scanning

Objective: Verify scanning of selected endpoint directories.

The endpoint scanner was executed using:

```
.\endpoint_scan.ps1
```

The script scanned locations including:

```
Downloads
Desktop
Documents
```

Actual Result:

The scanner reported:

```
Suspicious indicators found.
```

for the tested locations.

The endpoint report was successfully generated.

Status: PASS

17.13 Test Case 10 — Threat Hunting

Objective: Verify recursive scanning of the designated threat-hunting directory.

The test environment contained:

```
ThreatHunt
├── Documents
├── Downloads
├── Temp
└── USB
```

The threat-hunting script was used to search these locations for predefined indicators.

Expected Result:

Matching controlled indicators should be identified and recorded.

Actual Result:

The threat-hunting process completed and generated:

```
threat_hunting_report.txt
```

Status: PASS

17.14 Test Case 11 — Incident Response

Objective: Verify the incident-response evidence collection workflow.

The incident-response component was designed to record relevant information about detected files.

The collected information includes:

- Detection rule.
- File path.
- Detection timestamp.
- SHA-256 hash.

The generated report was stored as:

```
incident_report.txt
```

Status: PASS

17.15 Test Case 12 — Report Generation

Objective: Verify that scanning activities produce persistent output reports.

The following reports were generated:

```
scan_results.txt
endpoint_report.txt
incident_report.txt
threat_hunting_report.txt
```

Expected Result:

The reports should be created in the `results` directory.

Actual Result:

The reports were successfully generated.

Status: PASS

17.16 Test Summary

Test Case	Function Tested	Result
TC-01	YARA execution	PASS
TC-02	Basic rule detection	PASS
TC-03	Clean-file validation	PASS
TC-04	PowerShell indicator detection	PASS
TC-05	Command indicator detection	PASS
TC-06	Script indicator detection	PASS
TC-07	Recursive scanning	PASS
TC-08	Automated scanner	PASS
TC-09	Endpoint scanning	PASS
TC-10	Threat hunting	PASS
TC-11	Incident response	PASS
TC-12	Report generation	PASS

Overall Testing Result

All implemented project functions tested during the controlled laboratory evaluation produced the expected results.

The final automated scan successfully processed **6 files**, generated **5 rule matches**, and completed in approximately **0.056 seconds**.

The testing therefore confirms that the implemented prototype performs its intended rule-based detection, recursive scanning, automation, reporting, endpoint-scanning, threat-hunting, and incident-response demonstration functions.

18. RESULTS AND ANALYSIS

The implemented Simple Malware Scanner Using YARA was tested in a controlled Windows environment using predefined YARA rules and controlled test artifacts. The testing evaluated the scanner's ability to identify matching indicators, avoid reporting a clean test file, recursively scan nested directories, automate scanning through PowerShell, and generate security reports.

18.1 Final Scanner Results

The final execution of the main scanner was performed using:

```
.\scanner.ps1
```

The scanner produced the following summary:

```
=======================================
 SIMPLE MALWARE SCANNER USING YARA
=======================================

Scan Started : 31-08-2026 12:47:43
Target Folder: C:\Users\ubaid\YARA-Project\samples

========== SCAN SUMMARY ==========
Total Files Scanned : 6
Rule Matches        : 5
```

```
Scan Duration       : 0.056 Seconds
================================
```

The results demonstrate that the scanner successfully processed all six files in the test directory.

18.2 Detection Results

The following rule matches were generated:

Detection Rule	Detected File
Suspicious_PowerShell_Indicator	powershell_test.txt
Suspicious_Command_Indicator	command_test.txt
Test_Malware_Detection	nested\level2\deep_test.txt
Test_Malware_Detection	test_sample.txt
Suspicious_Script_Indicator	script_test.txt

The results demonstrate that the scanner correctly associated the controlled indicators with their corresponding YARA rules.

18.3 Clean File Analysis

The project included:

```
clean_sample.txt
```

This file did not contain an indicator matching the configured YARA rules.

No detection was generated for the clean sample.

This is an important result because it demonstrates basic negative testing: the scanner did not simply report every scanned file as suspicious.

18.4 Recursive Scanning Analysis

The project included a nested test file:

```
samples
└── nested
    └── level2
        └── deep_test.txt
```

The recursive scan successfully identified this file.

The result was:

```
Test_Malware_Detection
C:\Users\ubaid\YARA-Project\samples\nested\level2\deep_test.txt
```

This confirms that recursive scanning was successfully implemented and that files located below the primary target directory could be analyzed.

18.5 Performance Analysis

The final automated scan reported:

```
Total Files Scanned : 6
Scan Duration       : 0.056 Seconds
```

The measured duration demonstrates rapid scanning for the small controlled test dataset.

However, this measurement should not be interpreted as an enterprise-scale performance benchmark because only six files were scanned. Larger datasets, different file sizes, storage devices, and system configurations would produce different results.

18.6 Endpoint Scanning Results

The endpoint scanning component was tested against selected directories, including:

```
Downloads
Desktop
Documents
```

The scanner reported suspicious indicators in the tested locations and generated:

```
endpoint_report.txt
```

This demonstrates that the YARA-based detection logic can be applied beyond the project's dedicated sample directory.

18.7 Threat Hunting Results

The threat-hunting environment was organized into simulated locations:

```
ThreatHunt
├── Documents
├── Downloads
├── Temp
└── USB
```

The threat-hunting process recursively examined these locations for predefined indicators.

The resulting report was stored as:

```
threat_hunting_report.txt
```

This demonstrates how the same YARA detection mechanism can be used as part of a basic threat-hunting workflow.

18.8 Incident Response Results

The incident-response component generated an investigation report containing information associated with detected artifacts.

The report was stored as:

```
incident_report.txt
```

The incident-response workflow incorporates information such as:

- Detection rule.
- File path.
- Detection timestamp.
- SHA-256 hash.

This information can assist analysts in documenting and tracking suspicious artifacts during an investigation.

18.9 Report Generation Results

The project successfully generated separate reports for its major security functions:

```
results
├── endpoint_report.txt
├── incident_report.txt
├── scan_results.txt
└── threat_hunting_report.txt
```

Separating the reports allows the results of malware scanning, endpoint scanning, threat hunting, and incident response to be reviewed independently.

18.10 Overall Results

The overall implementation produced the following outcomes:

Function	Result
YARA execution	Successful
Custom rule detection	Successful
Clean-file validation	Successful
Recursive scanning	Successful
PowerShell automation	Successful
Scan statistics	Successful
Endpoint scanning	Successful
Threat hunting	Successful
Incident-response workflow	Successful
SHA-256 evidence collection	Implemented
Report generation	Successful

18.11 Interpretation of Results

The test results demonstrate that the project successfully implements a basic rule-based file detection workflow using YARA and PowerShell.

The scanner was able to:

1. Scan a defined directory.
2. Process multiple files.
3. Identify files containing configured test indicators.
4. Report the corresponding YARA rule.
5. Search nested directories.
6. Distinguish a clean test file from matching test artifacts.
7. Measure scan duration.
8. Generate persistent reports.
9. Extend the scanning workflow to endpoint and threat-hunting scenarios.
10. Support basic incident-response evidence collection.

The project therefore meets its primary technical objective of demonstrating a simple, customizable YARA-based malware detection workflow.

18.12 Important Detection Limitation

A YARA match indicates that the conditions defined in a rule were satisfied. It does **not automatically prove that the matched file is malware**.

For this project, the test artifacts were intentionally created to trigger specific rules. Therefore, the appropriate interpretation is that the scanner identified **potentially suspicious or matching artifacts according to the configured YARA rules**.

In a real-world investigation, a YARA match should be followed by additional analysis, such as file inspection, behavioral analysis, sandboxing, reputation checks, or other appropriate security investigation techniques.

18.13 Result Summary

The final project successfully demonstrated:

```
6 files scanned
       ↓
5 YARA rule matches
       ↓
1 clean test file without a match
       ↓
Nested directory successfully scanned
       ↓
Endpoint scanning demonstrated
       ↓
Threat hunting demonstrated
       ↓
Incident-response evidence workflow implemented
```

```
        ↓
Reports generated
```

The results confirm that the Simple Malware Scanner Using YARA provides a functional prototype for controlled malware-detection, threat-hunting, and security-analysis activities.

19. SCREENSHOTS AND EVIDENCE

This section presents the screenshots and supporting evidence collected during the implementation and testing of the Simple Malware Scanner Using YARA.

The screenshots demonstrate the installation and validation of YARA, rule creation, controlled sample detection, recursive scanning, automated scanning, endpoint scanning, threat hunting, incident-response processing, and report generation.

All screenshots should represent the actual project environment and results obtained during testing.

19.1 YARA Installation and Validation

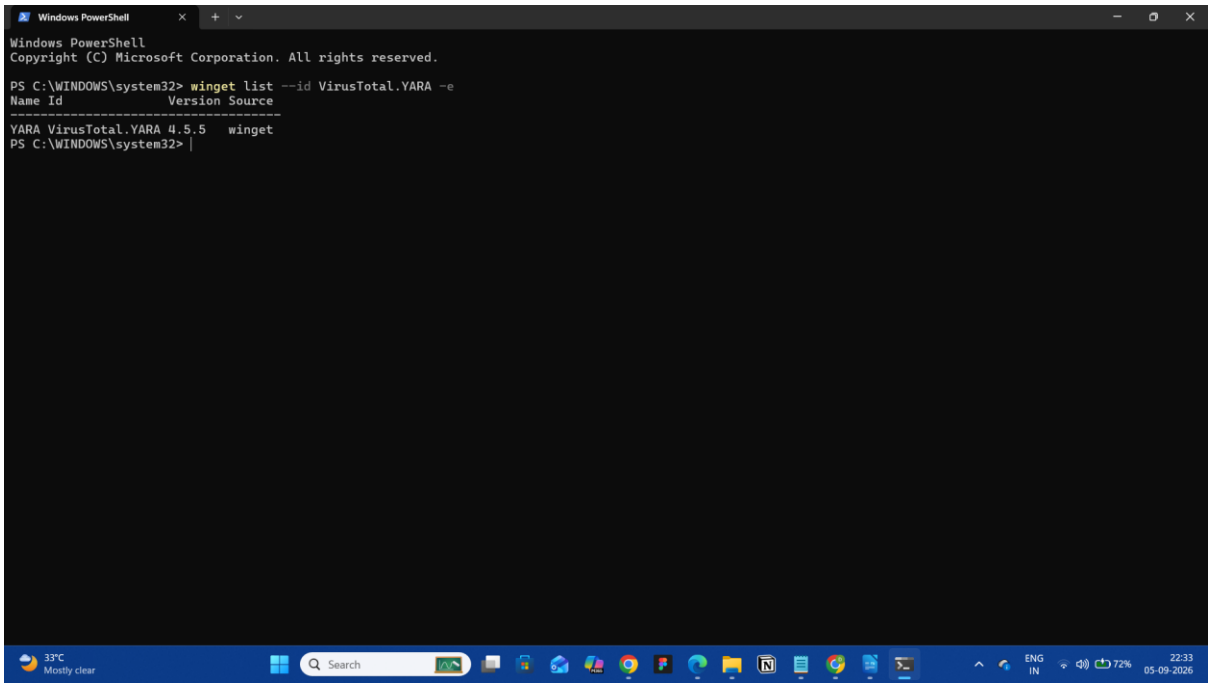


Figure 21.1: YARA Package Verification

Command:

```
winget list --id VirusTotal.YARA -e
```

Purpose:

This screenshot demonstrates that the YARA package was recognized as installed on the Windows system.

Observation:

The command displayed the installed YARA package and its version.

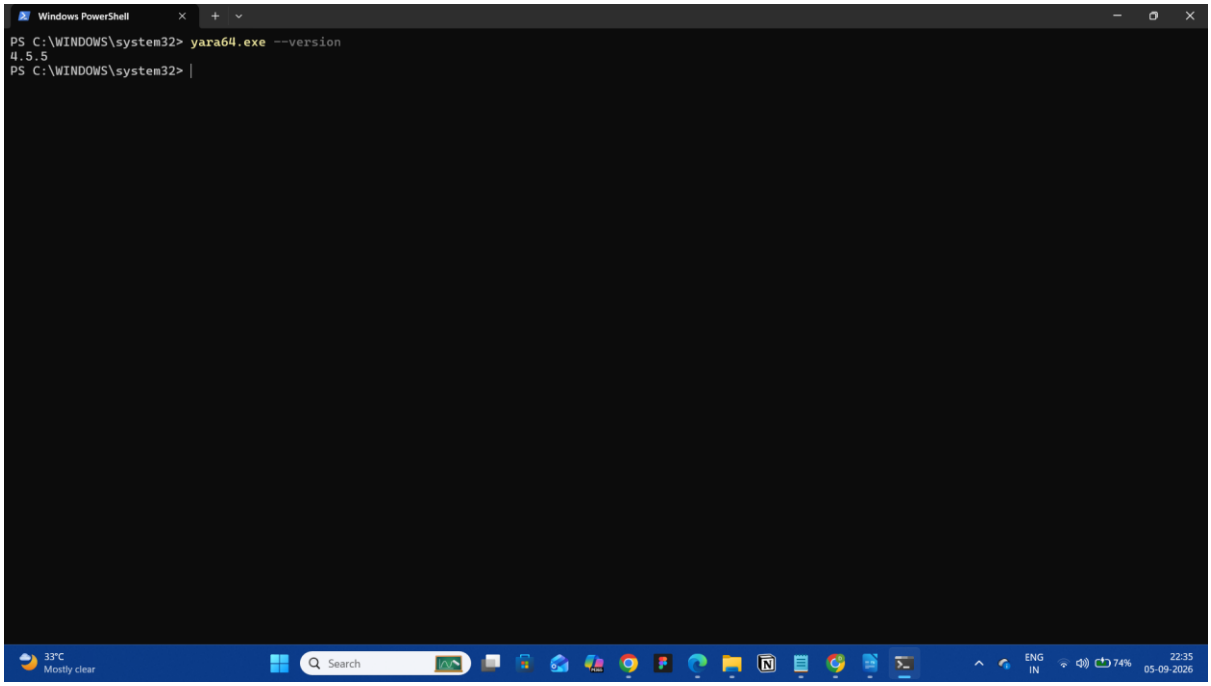


Figure 21.2: YARA Executable Validation

Command:

```
yara64.exe
```

Purpose:

To verify that the YARA executable can be executed successfully.

Observation:

YARA displayed its command-line usage information, confirming that the executable was functioning.

19.2 Project Directory Structure

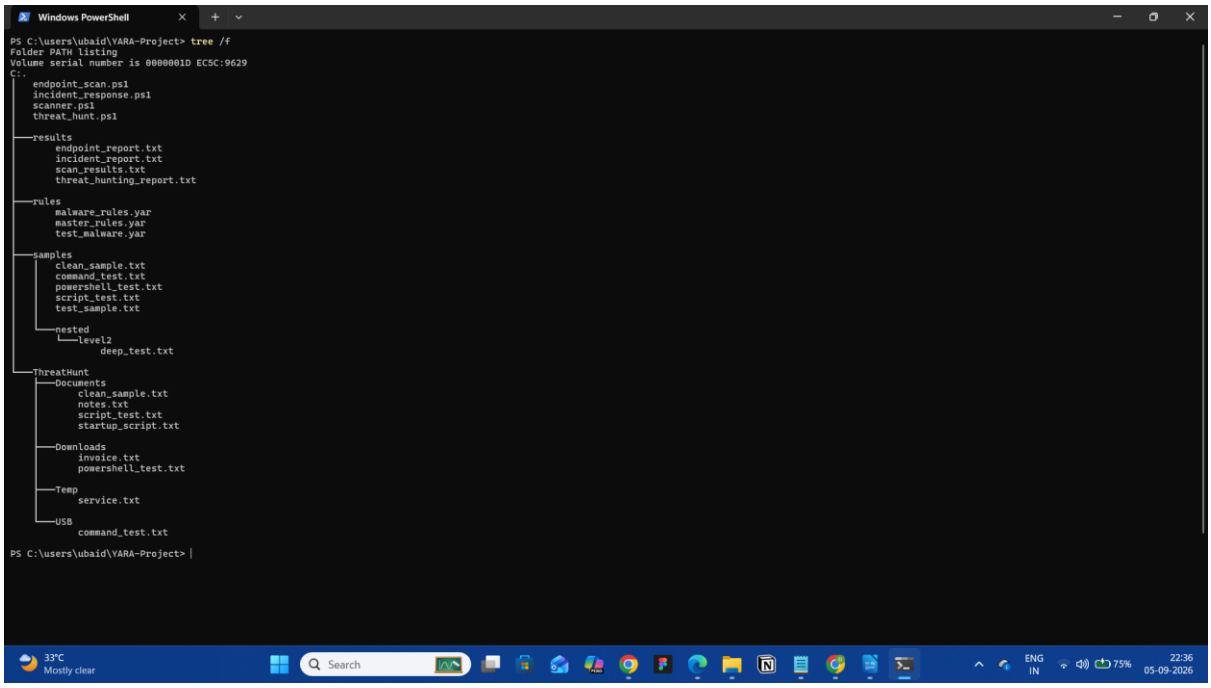


Figure 21.3: Project Directory Structure

Command:

```
tree /F
```

Purpose:

To display the complete organization of the project.

Observation:

The project contains separate directories for rules, samples, threat-hunting data, results, and PowerShell scripts.

The structure includes:

```
YARA-Project
├── rules
├── samples
├── ThreatHunt
├── results
├── scanner.ps1
├── endpoint_scan.ps1
├── incident_response.ps1
└── threat_hunt.ps1
```

19.3 YARA Rule Evidence

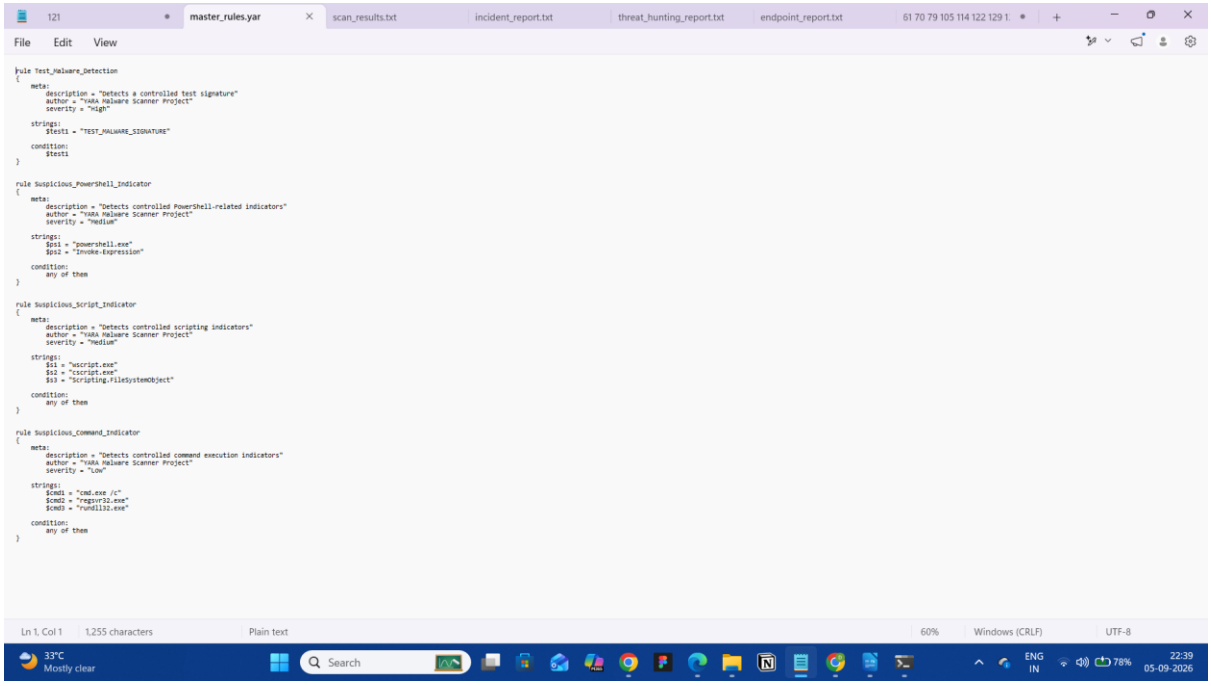


Figure 21.4: Custom YARA Rules

Evidence:

Screenshot of the YARA rule file opened in the text editor.

Purpose:

To demonstrate the customized rule definitions used by the scanner.

Observation:

The rule file contains detection logic for the controlled indicators used during testing.

19.4 Individual File Detection

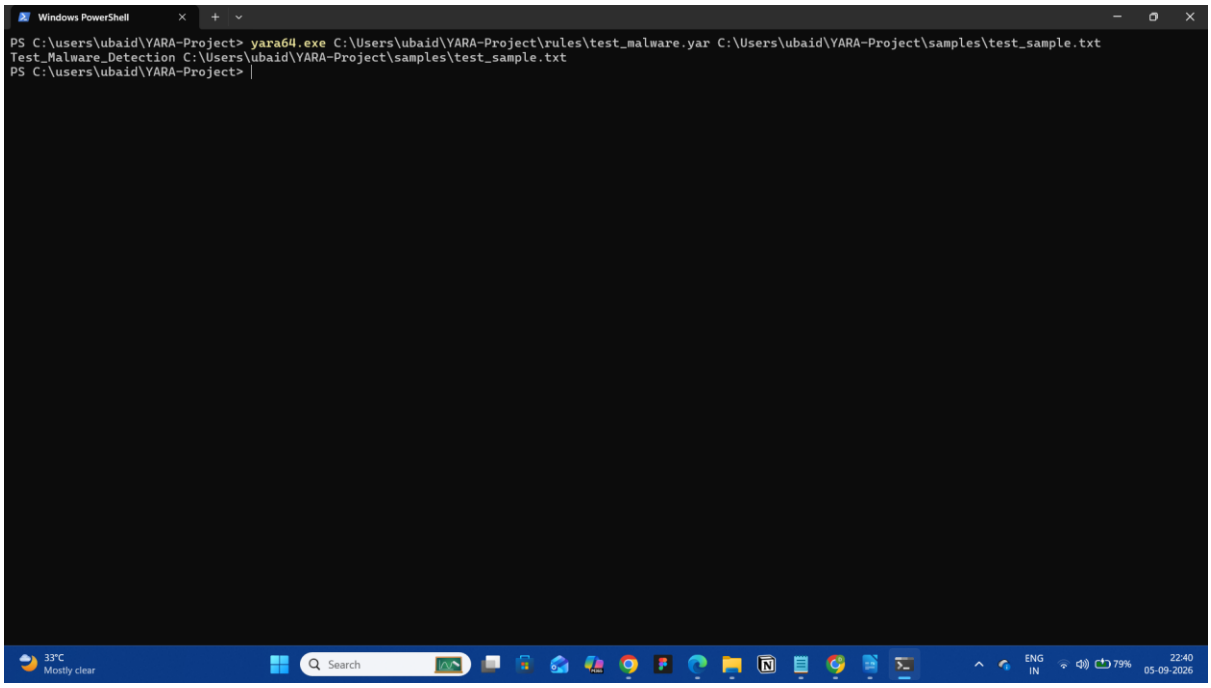


Figure 21.5: Basic YARA Detection

Command:

```
yara64.exe C:\Users\ubaid\YARA-Project\rules\test_malware.yar
C:\Users\ubaid\YARA-Project\samples\test_sample.txt
```

Observed Result:

```
Test_Malware_Detection
```

Purpose:

To verify that a YARA rule successfully detects its corresponding controlled test indicator.

Result:

PASS

19.5 Clean File Validation

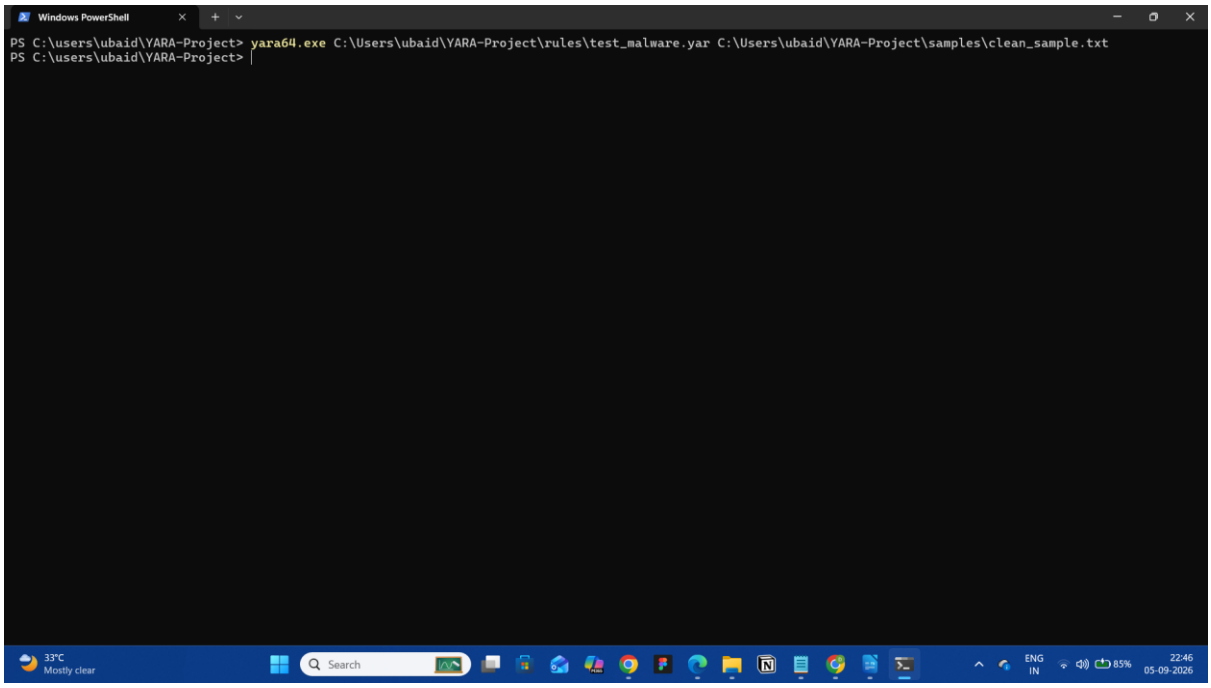


Figure 21.6: Clean File Test

Test File:

`clean_sample.txt`

Purpose:

To verify that a clean test artifact does not generate a YARA match.

Observed Result:

No rule match was returned.

Result:

PASS

19.6 Individual Detection Rules

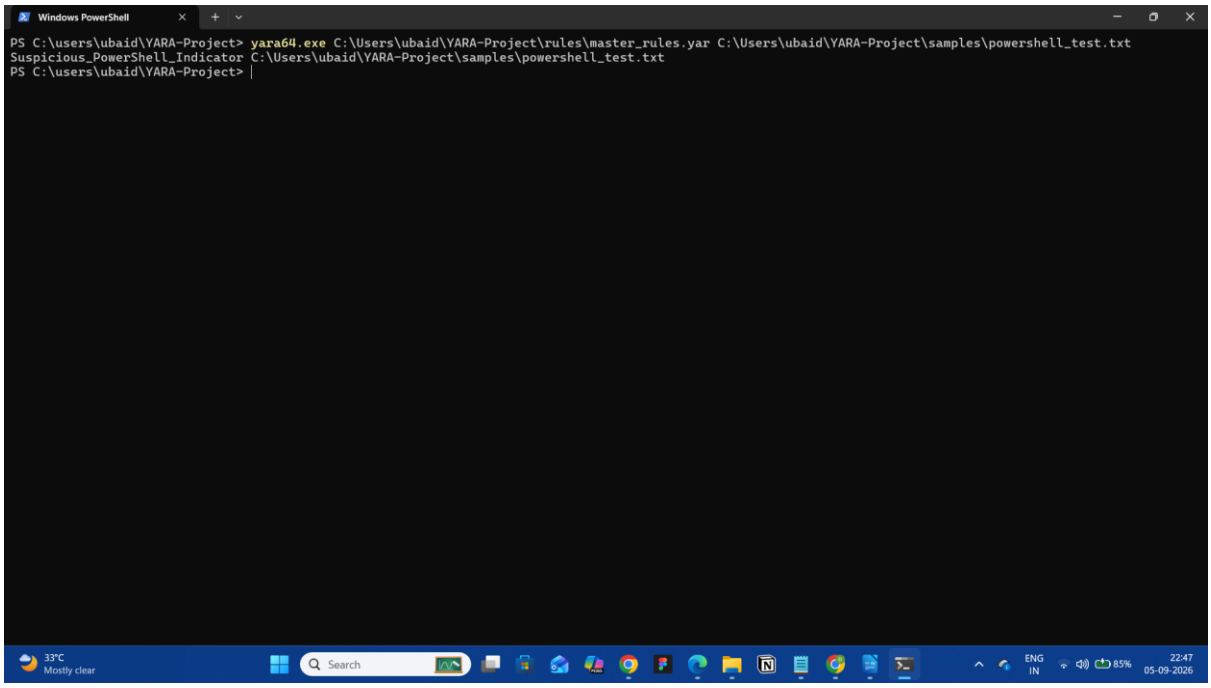


Figure 21.7: PowerShell Indicator Detection

Observed Rule:

`Suspicious_PowerShell_Indicator`

Test File:

`powershell_test.txt`

Result:

PASS

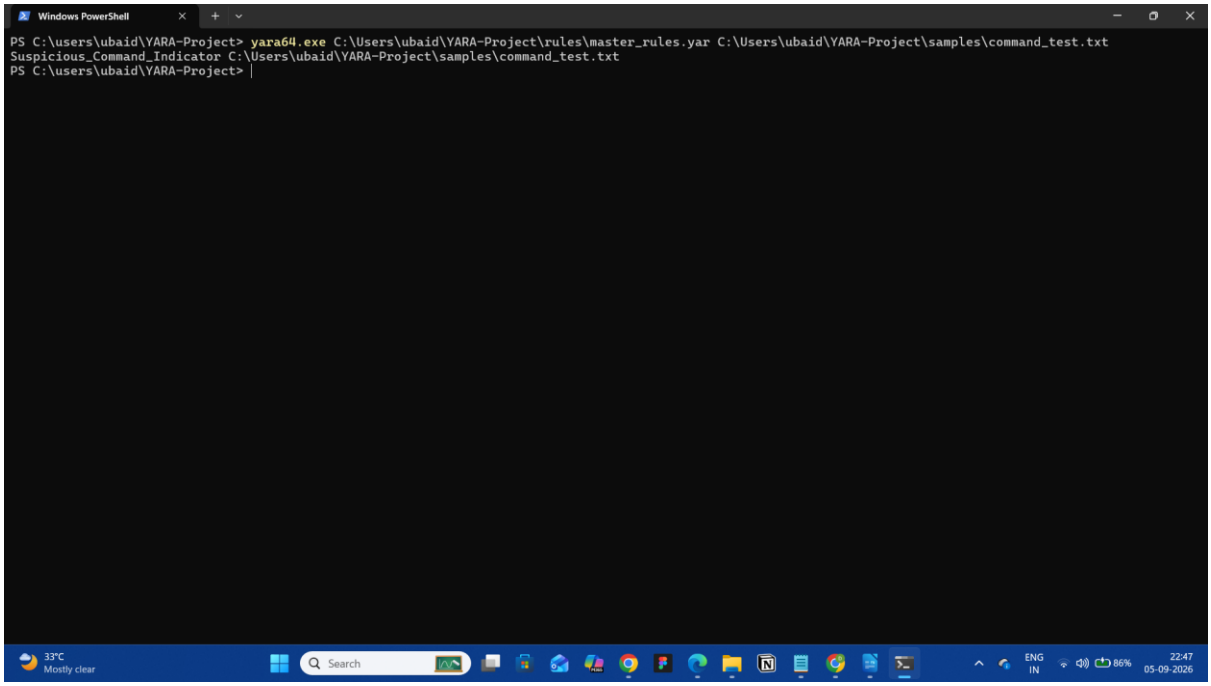


Figure 21.8: Command Indicator Detection

Observed Rule:

`Suspicious_Command_Indicator`

Test File:

`command_test.txt`

Result:

PASS

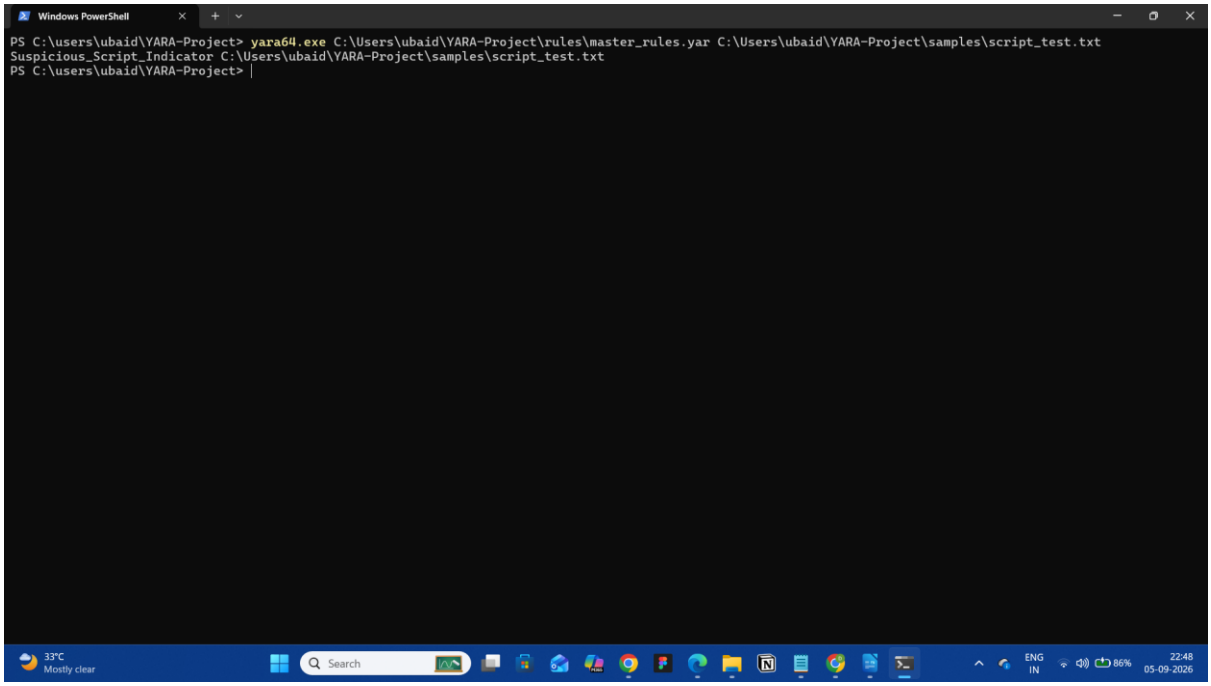


Figure 21.9: Script Indicator Detection

Observed Rule:

`Suspicious_Script_Indicator`

Test File:

`script_test.txt`

Result:

PASS

19.7 Recursive Scanning Evidence

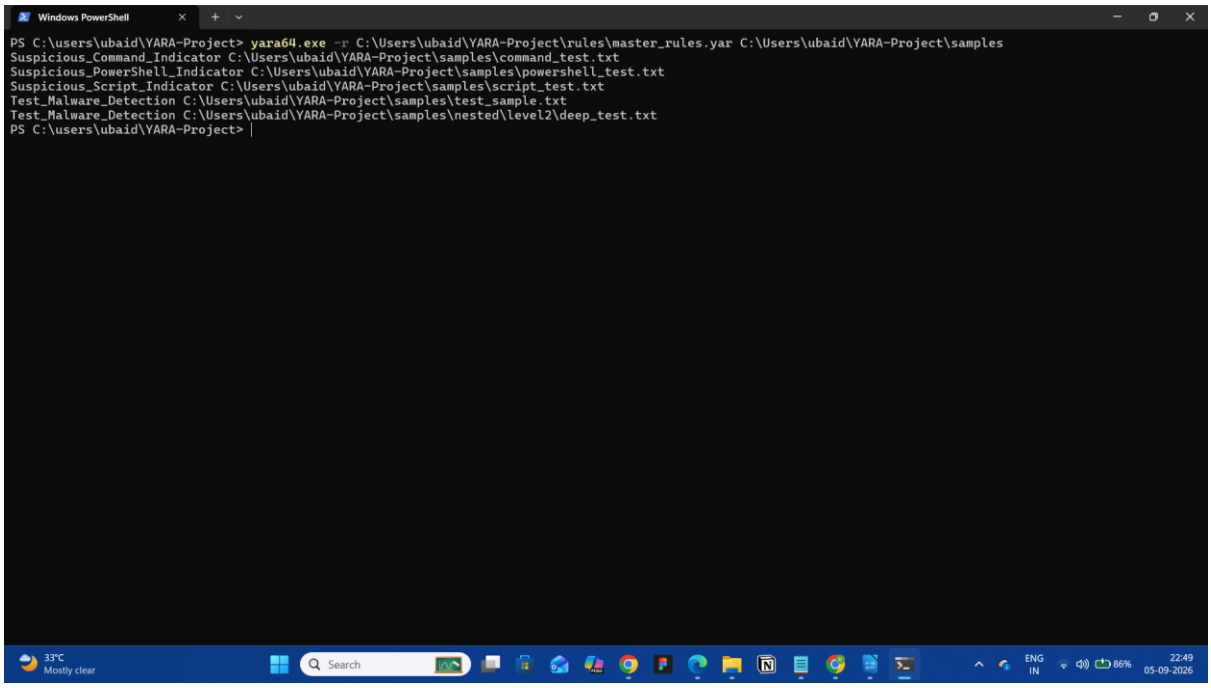


Figure 21.10: Recursive Directory Scan

Command:

```
yara64.exe -r C:\Users\ubaid\YARA-Project\rules\master_rules.yar
C:\Users\ubaid\YARA-Project\samples
```

Purpose:

To demonstrate scanning of the target directory and its nested subdirectories.

Observed Result:

```
Test_Malware_Detection C:\Users\ubaid\YARA-
Project\samples\nested\level2\deep_test.txt
```

Result:

PASS

This confirms that the scanner successfully detected a controlled test artifact located several directory levels below the main scanning location.

19.8 Automated Scanner Evidence

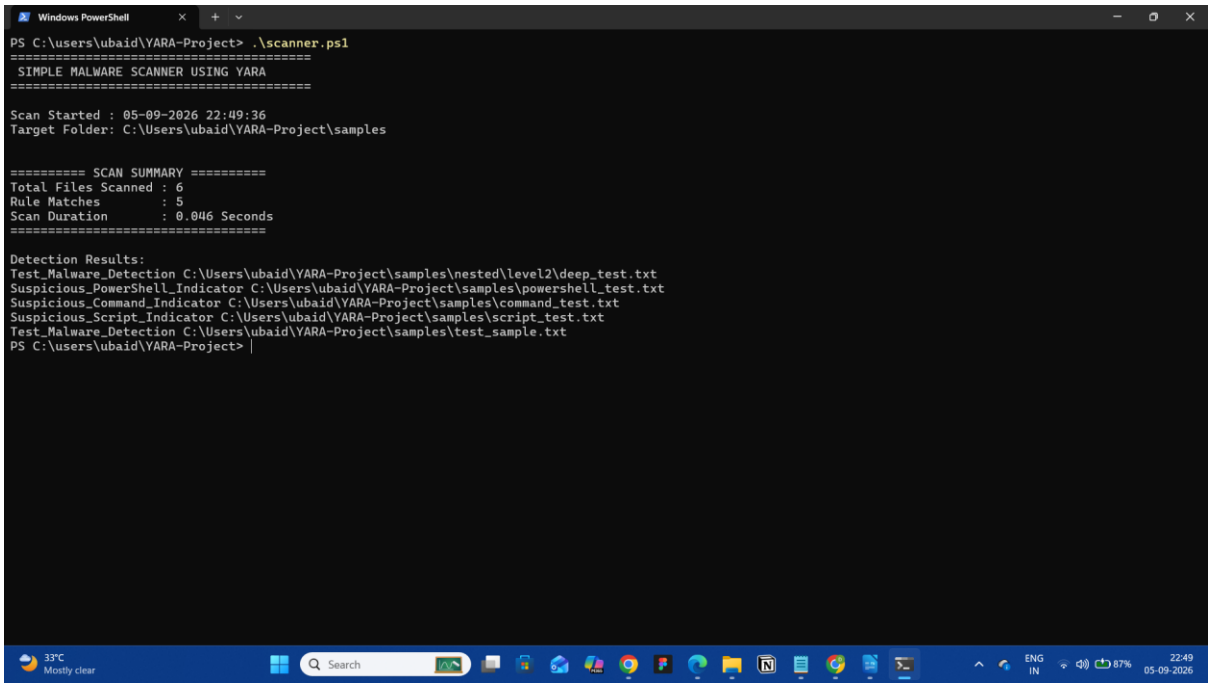


Figure 21.11: PowerShell Scanner Execution

Command:

```
.\scanner.ps1
```

Purpose:

To demonstrate execution of the automated malware scanner.

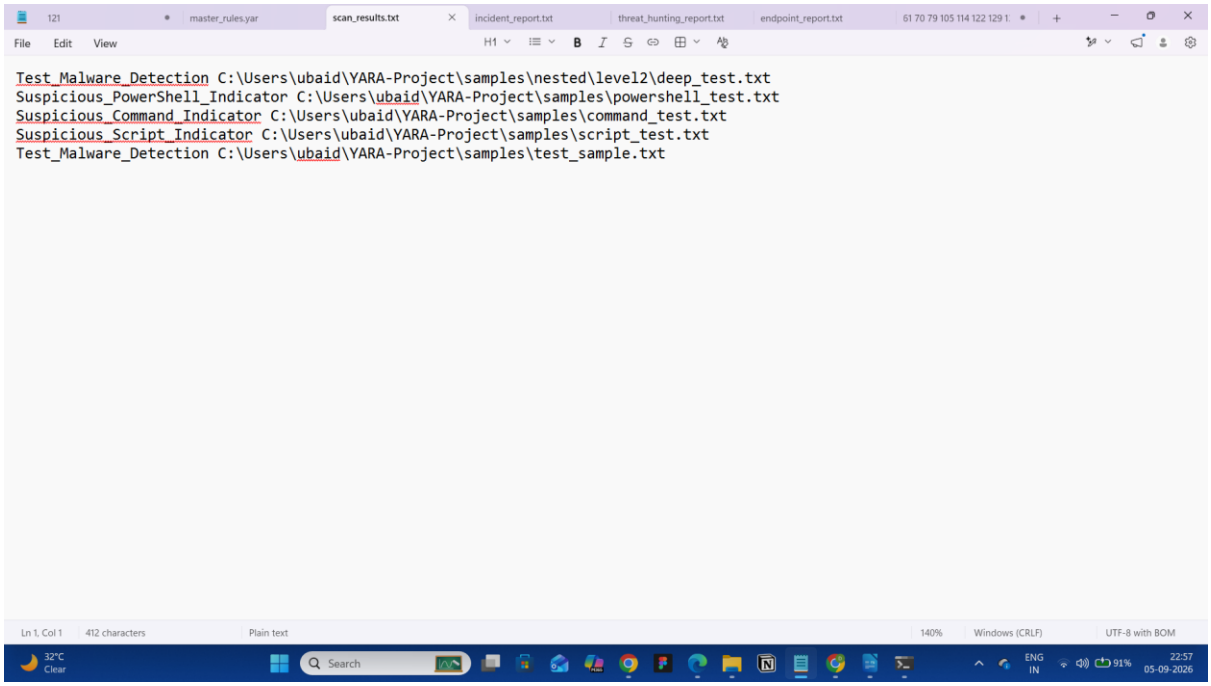


Figure 21.12: Final Scanner Summary

Observed Result:

```
Total Files Scanned : 6
Rule Matches        : 5
Scan Duration       : 0.056 Seconds
```

Purpose:

To demonstrate the final scanning statistics generated by the project.

Result:

PASS

19.9 Endpoint Scanning Evidence

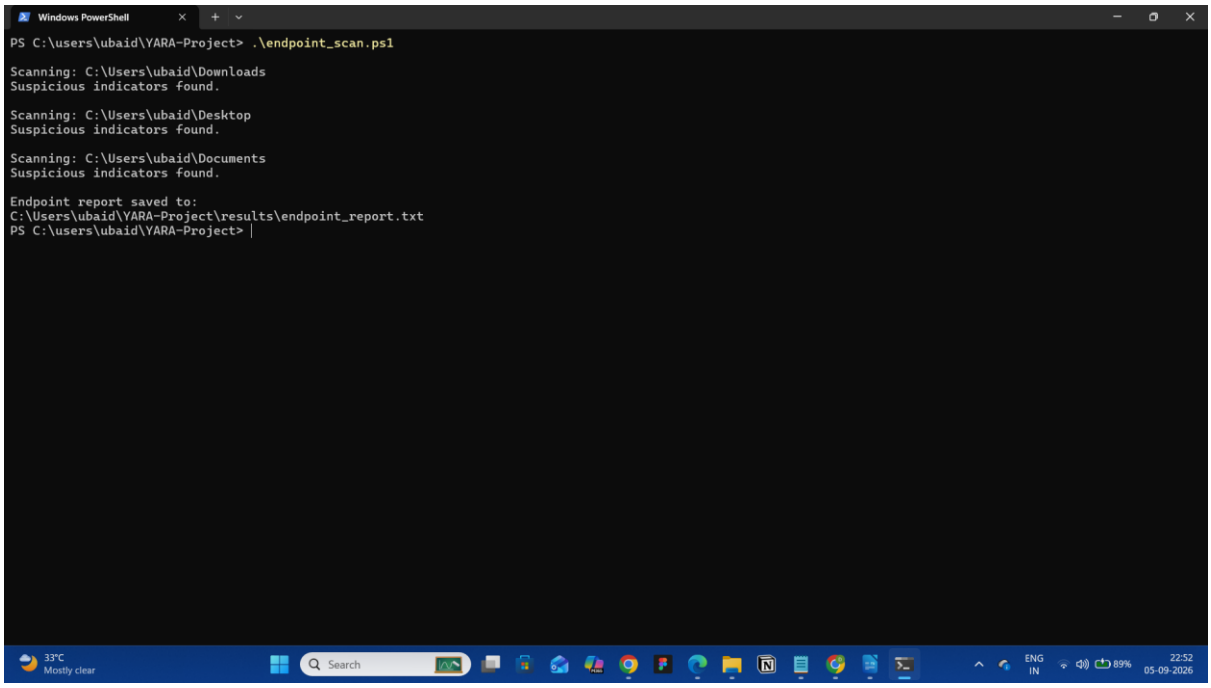


Figure 21.13: Endpoint Scanner Execution

Command:

```
.\endpoint_scan.ps1
```

Purpose:

To demonstrate scanning of selected endpoint directories.

Observed Locations:

```
Downloads
Desktop
Documents
```

Observed Result:

```
Suspicious indicators found.
```

Report Generated:

```
endpoint_report.txt
```

Result:

PASS

19.10 Threat Hunting Evidence

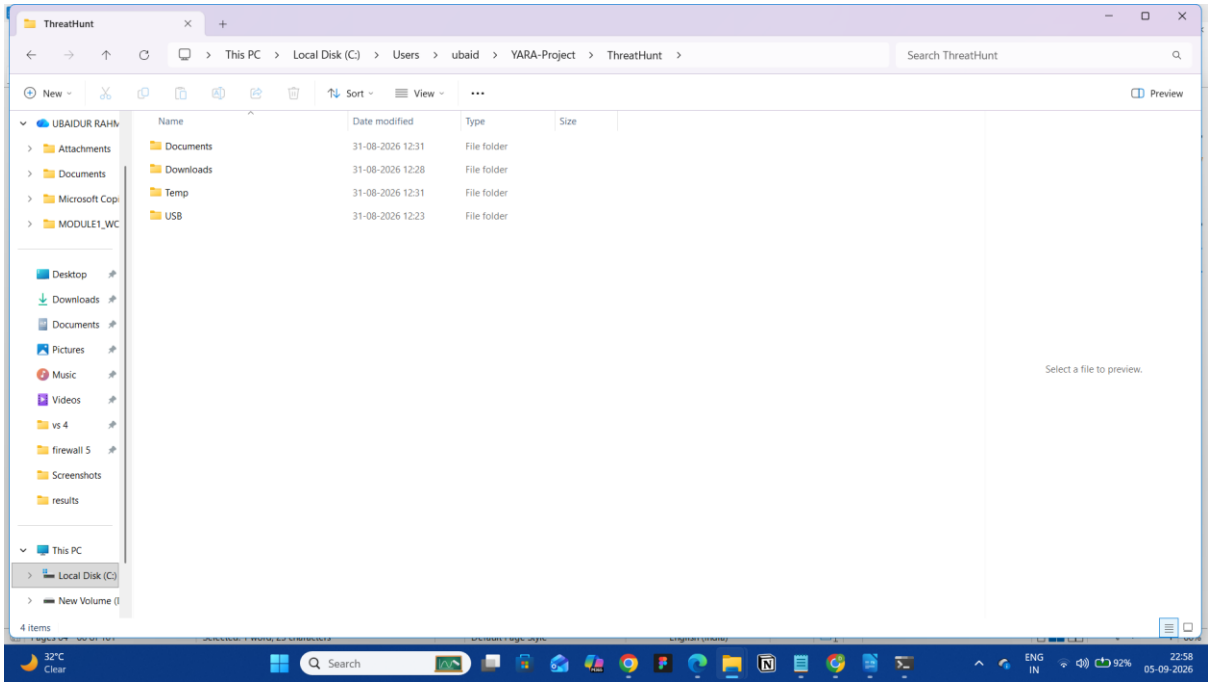


Figure 21.14: Threat Hunting Directory

Directory:

```
ThreatHunt
├── Documents
├── Downloads
├── Temp
└── USB
```

Purpose:

To demonstrate the simulated locations used for the threat-hunting exercise.

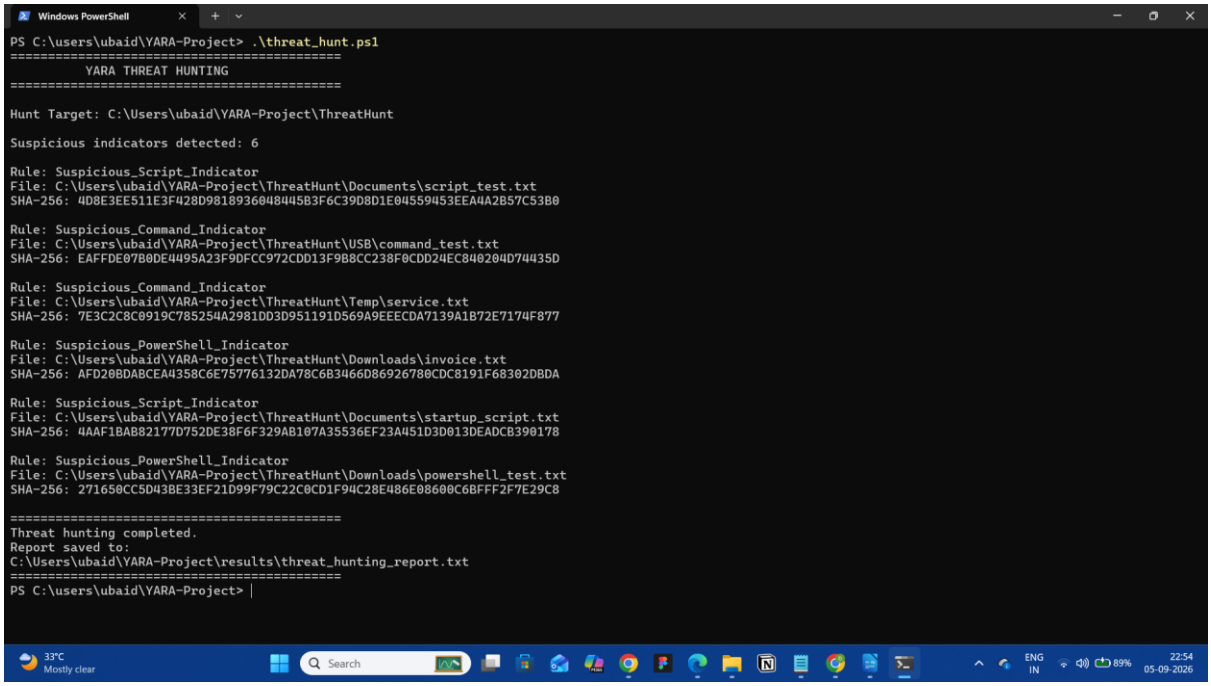


Figure 21.15: Threat Hunting Execution

Evidence:

Screenshot showing execution of:

`.\threat_hunt.ps1`

Purpose:

To demonstrate recursive searching of the designated threat-hunting environment.

Report Generated:

`threat_hunting_report.txt`

Result:

PASS

19.11 Incident Response Evidence

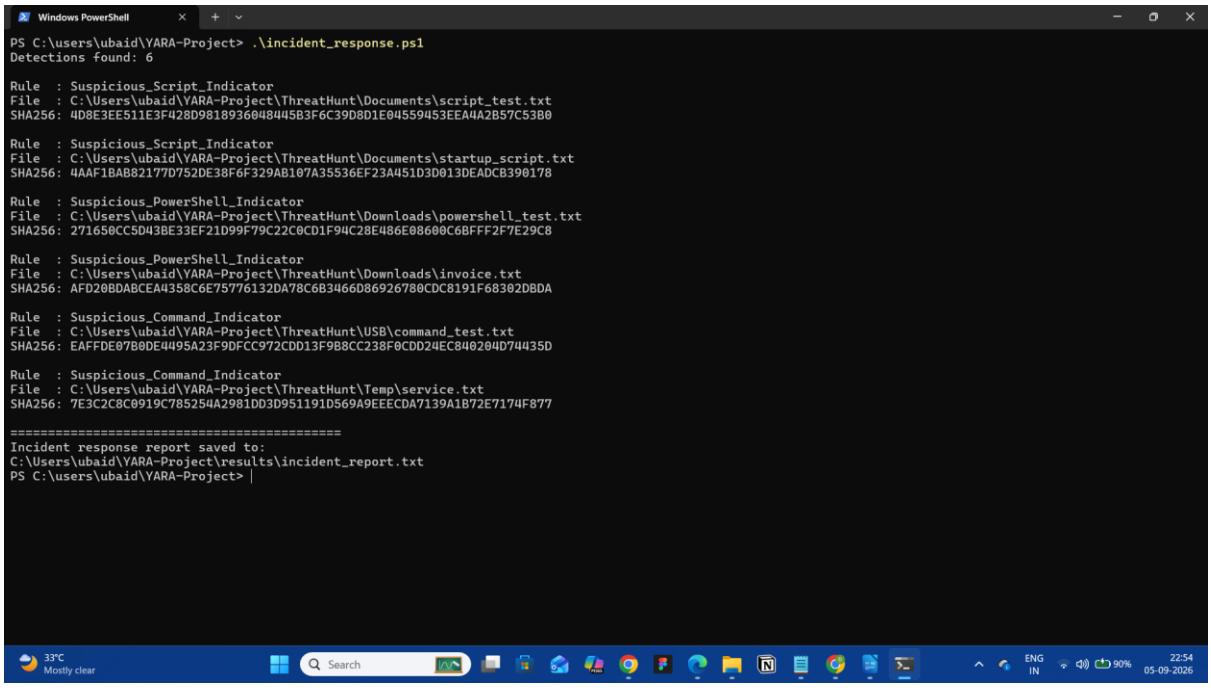


Figure 21.16: Incident Response Execution

Evidence:

Screenshot showing execution of:

`.\incident_response.ps1`

Purpose:

To demonstrate the incident-response evidence collection workflow.

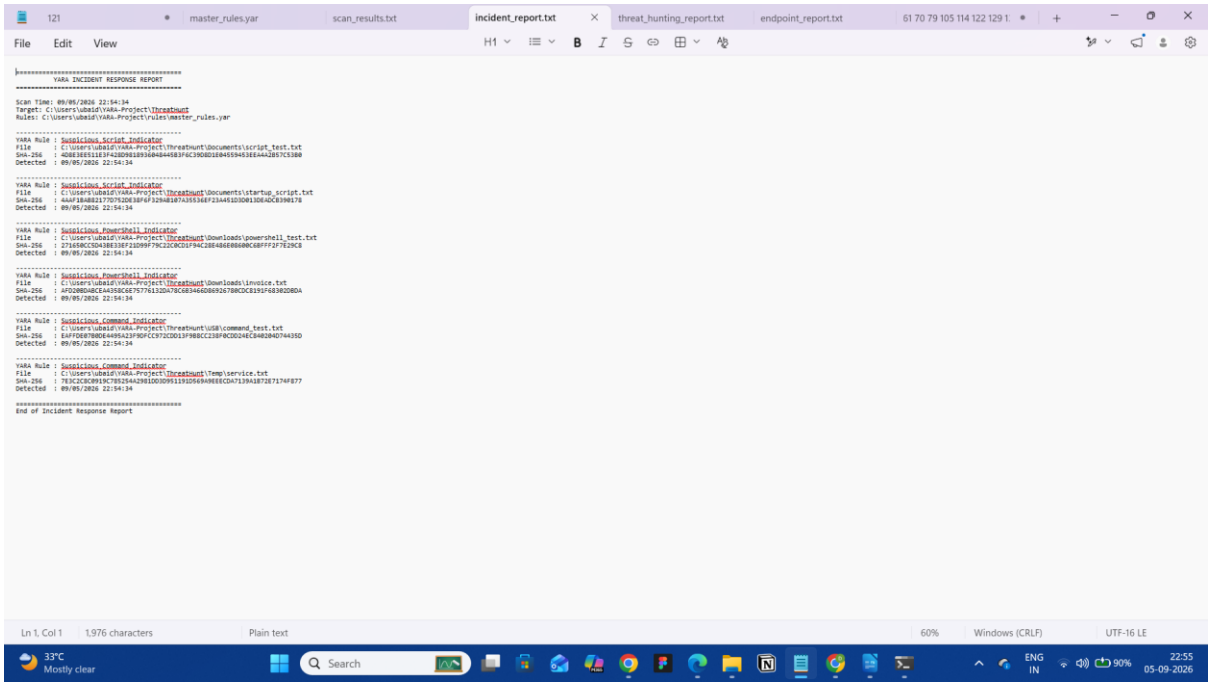


Figure 21.17: Incident Response Report

Report:

`incident_report.txt`

The report records information associated with detected artifacts, including relevant file information and SHA-256 evidence.

Result:

PASS

19.12 Generated Reports

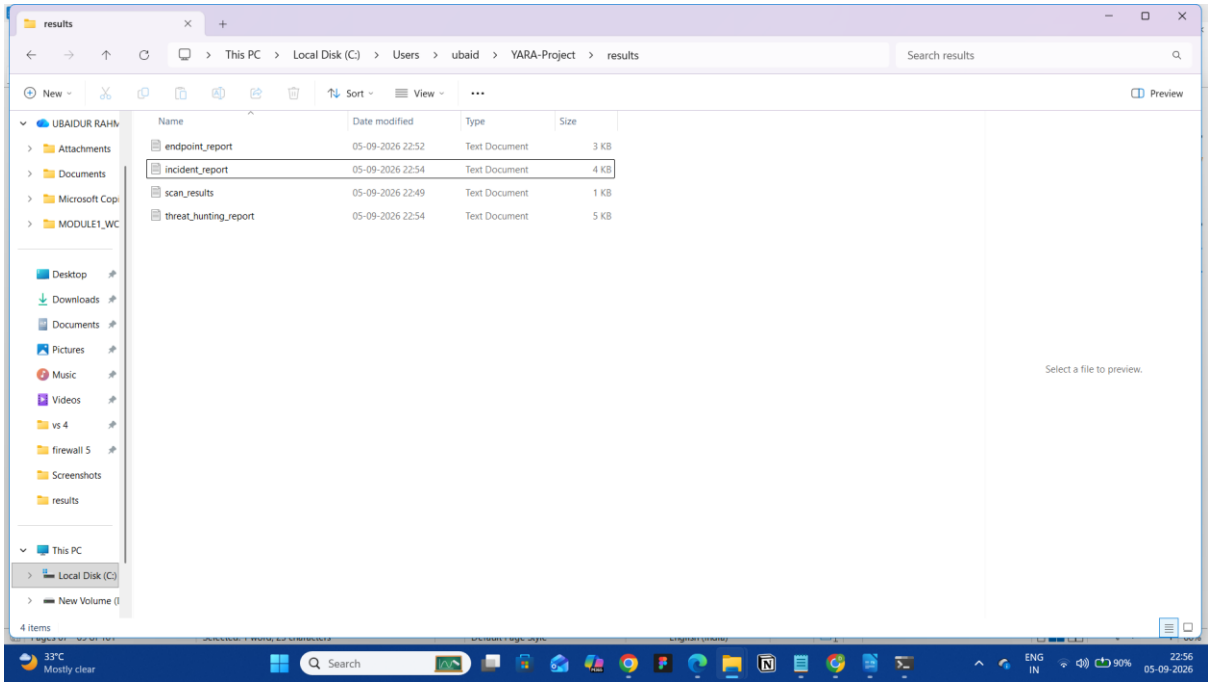


Figure 21.18: Results Directory

The results directory contains:

```
results
├── endpoint_report.txt
├── incident_report.txt
├── scan_results.txt
└── threat_hunting_report.txt
```

Purpose:

To demonstrate that the project successfully generates persistent output reports for its different security functions.

Result:

PASS

19.13 Evidence Summary

The screenshots and supporting evidence demonstrate the following completed functions:

Evidence	Function	Status
YARA package verification	Installation	PASS
YARA executable output	Validation	PASS
YARA rule screenshot	Rule development	PASS
Individual detection	Basic scanning	PASS

Evidence	Function	Status
Clean sample	Negative testing	PASS
Recursive scan	Recursive detection	PASS
Scanner output	Automation	PASS
Endpoint output	Endpoint scanning	PASS
Threat-hunting output	Threat hunting	PASS
Incident report	Incident response	PASS
Results directory	Report generation	PASS

The collected evidence provides visual confirmation of the major implementation and testing activities performed during the project.

19.14 Evidence Handling Note

The screenshots and reports included in this project document results obtained from a controlled laboratory environment.

The test artifacts were specifically created to trigger the configured YARA rules. A YARA match indicates that a rule condition was satisfied; it does not independently establish that the matched artifact is malicious.

For real-world investigations, YARA detections should be validated using additional analysis and appropriate security investigation procedures.

20. ADVANTAGES OF THE PROPOSED SYSTEM

The Simple Malware Scanner Using YARA provides a lightweight and customizable approach to file-based threat detection. By combining YARA's pattern-matching capabilities with PowerShell automation, the proposed system provides a practical solution for controlled malware-analysis, endpoint-scanning, and threat-hunting activities.

The major advantages of the proposed system are described below.

20.1 Lightweight Detection

YARA is a lightweight detection engine that can be executed from the command line without requiring a large security-management platform.

This makes the proposed system suitable for:

- Cybersecurity laboratories.
- Malware-analysis exercises.
- Security testing environments.
- Small-scale file investigations.
- Educational demonstrations.

The project demonstrated rapid scanning of the controlled test dataset, completing the final six-file scan in approximately 0.056 seconds.

20.2 Customizable Detection Rules

One of the primary advantages of YARA is the ability to create customized detection rules.

The project includes rules for controlled indicators such as:

```
Suspicious_PowerShell_Indicator
Suspicious_Command_Indicator
Suspicious_Script_Indicator
Test_Malware_Detection
```

Security analysts can modify existing rules or create new rules when additional indicators need to be investigated.

20.3 Recursive Directory Scanning

The proposed system supports recursive scanning of directories.

The project demonstrated this capability using a nested test artifact:

```
samples
```

```
└── nested
    └── level2
        └── deep_test.txt
```

The scanner successfully identified the matching indicator within the nested directory.

This capability is useful when files may be distributed across multiple directory levels.

20.4 Automation Using PowerShell

PowerShell was used to automate repetitive scanning and security-analysis tasks.

The project contains:

```
scanner.ps1
endpoint_scan.ps1
threat_hunt.ps1
incident_response.ps1
```

Automation reduces manual effort and provides a consistent procedure for executing security tasks.

20.5 File-Level Visibility

The scanner reports the YARA rule that matched together with the associated file path.

For example:

```
Suspicious_PowerShell_Indicator
C:\Users\ubaid\YARA-Project\samples\powershell_test.txt
```

This provides analysts with useful information about where the configured indicator was identified.

20.6 Support for Endpoint Security Activities

The endpoint scanning component allows the same YARA detection mechanism to be applied to selected endpoint directories.

The project demonstrated scanning of locations such as:

```
Downloads
Desktop
Documents
```

This provides a basic additional security layer for file inspection.

20.7 Threat Hunting Capability

The proposed system can be used as a basic threat-hunting tool by applying YARA rules to selected locations.

The project created a simulated threat-hunting environment containing:

```
ThreatHunt
├── Documents
├── Downloads
├── Temp
└── USB
```

The threat-hunting script recursively searches these locations for configured indicators.

20.8 Incident Response Support

The system provides basic support for incident-response activities.

When a relevant artifact is identified, information such as the file path, detection rule, timestamp, and SHA-256 hash can be recorded.

This helps maintain useful investigation information for further analysis.

20.9 Automated Report Generation

The proposed system generates reports for different security activities.

The project produces:

```
endpoint_report.txt
incident_report.txt
scan_results.txt
threat_hunting_report.txt
```

Automated report generation improves documentation and allows security results to be reviewed after scanning has completed.

20.10 Easy to Extend

The proposed architecture allows additional YARA rules and scanning functions to be added without completely redesigning the system.

For example, future versions could include:

- Additional YARA rule categories.
- File-type-specific rules.
- Hash-based indicators.
- Automated scheduling.
- Centralized logging.
- Additional endpoint locations.
- Integration with security monitoring systems.

20.11 Educational Value

The project provides practical experience with several cybersecurity concepts, including:

- Malware detection.
- Signature and pattern-based analysis.
- YARA rule development.
- PowerShell scripting.
- Recursive file scanning.
- Threat hunting.
- Incident response.
- File hashing.
- Security reporting.

This makes the system particularly useful as a cybersecurity training and laboratory project.

20.12 Cost-Effective Approach

The proposed system uses lightweight tools and locally maintained rules rather than requiring an expensive enterprise security platform.

This makes it suitable for learning environments and proof-of-concept implementations where resources may be limited.

20.13 Overall Advantage

The major advantage of the proposed system is the combination of **customizable YARA detection with PowerShell automation**.

The project demonstrates that a relatively simple architecture can provide multiple security capabilities, including:

```
YARA Rules
     ↓
File Scanning
     ↓
Recursive Detection
     ↓
Endpoint Scanning
     ↓
Threat Hunting
     ↓
Incident Response
     ↓
Report Generation
```

Therefore, the proposed system provides a practical foundation for learning and demonstrating rule-based malware detection and security-analysis workflows.

21. LIMITATIONS

Although the Simple Malware Scanner Using YARA successfully demonstrates rule-based file detection, the proposed system has several limitations. Understanding these limitations is important when evaluating the system's effectiveness and considering its use in real-world security environments.

21.1 Dependence on YARA Rules

The effectiveness of the scanner depends heavily on the quality and coverage of its YARA rules.

If a suspicious file does not contain characteristics represented by the configured rules, the scanner may not identify it.

Therefore, the rule set must be regularly reviewed, tested, and updated.

21.2 Possible False Positives

YARA rules may match legitimate files when those files contain strings or patterns that satisfy the rule conditions.

For this reason, a YARA match should be treated as an indicator requiring further investigation rather than automatic proof that a file is malicious.

21.3 Possible False Negatives

The scanner may fail to identify a malicious artifact when its characteristics are not represented in the available rules.

Sophisticated or previously unknown threats may therefore require additional detection and analysis techniques.

21.4 Limited Test Dataset

The project was evaluated using a small controlled dataset.

The final scanner test processed:

```
Total Files Scanned : 6
Rule Matches        : 5
```

The measured duration was:

```
Scan Duration       : 0.056 Seconds
```

These results demonstrate successful operation within the project environment but should not be considered an enterprise-scale performance benchmark.

21.5 No Real-Time Monitoring

The current implementation performs scanning when the scripts are executed.

It does not continuously monitor file-system activity for newly created, modified, or downloaded files.

Real-time monitoring would require additional system-monitoring functionality.

21.6 No Behavioral Analysis

The proposed system primarily performs rule-based file analysis.

It does not independently monitor runtime behavior such as:

- Process execution.
- Network communication.
- Registry modifications.
- Memory activity.
- Persistence mechanisms.
- Runtime system changes.

Consequently, behavioral analysis would require additional security technologies.

21.7 No Automatic Quarantine or Removal

The current scanner reports matching files but does not automatically delete or quarantine them.

This is appropriate for the project's controlled environment because automatic removal could result in accidental modification or deletion of legitimate files.

A production implementation could introduce controlled remediation after appropriate analyst validation.

21.8 Limited Enterprise Management

The project is designed as a local proof-of-concept and does not include centralized management capabilities.

It does not currently provide:

- Centralized endpoint administration.
- Multi-system deployment.
- Centralized rule management.

- Enterprise dashboards.
- Distributed scanning.
- Role-based security administration.

21.9 Limited Security Monitoring Integration

The current implementation generates local text reports.

It does not directly integrate with enterprise security platforms such as SIEM, EDR, SOAR, or centralized log-management systems.

Such integrations would improve monitoring and incident correlation in larger environments.

21.10 Manual Rule Maintenance

Although the scanner automates execution, YARA rules currently require manual creation and maintenance.

Security analysts must review new indicators and update the rules accordingly.

Poorly designed rules may reduce detection accuracy or increase false positives.

21.11 Limited File Analysis

The project focuses primarily on identifying configured indicators within files.

It does not provide comprehensive static malware analysis capabilities such as:

- Portable Executable structure analysis.
- Disassembly.
- Reverse engineering.
- Automated sandbox execution.
- Advanced memory analysis.

These activities require specialized malware-analysis tools.

21.12 Controlled Laboratory Environment

The test artifacts used in this project were intentionally created or selected to trigger the configured rules.

Therefore, the demonstrated detections represent controlled testing rather than evidence of infections on a production system.

The project should be evaluated as a cybersecurity proof-of-concept and educational implementation.

21.13 Overall Limitation Assessment

The proposed system provides useful rule-based detection and automation capabilities, but it should not be considered a complete standalone malware-defense platform.

Its effectiveness can be improved by combining YARA with additional security controls such as endpoint monitoring, behavioral analysis, antivirus/EDR solutions, centralized logging, and continuous threat-intelligence updates.

The limitations identified in this chapter also provide the basis for the future enhancements discussed in the next chapter.

22. FUTURE ENHANCEMENTS

The current Simple Malware Scanner Using YARA provides a functional proof-of-concept for rule-based file detection, endpoint scanning, threat hunting, and basic incident-response activities. The system can be further enhanced to improve automation, detection capability, scalability, usability, and integration with other cybersecurity technologies.

The following enhancements are proposed for future versions of the project.

22.1 Real-Time File Monitoring

The scanner could be enhanced to continuously monitor selected directories for newly created or modified files.

When a new file is detected, the system could automatically submit it to the YARA scanning engine.

A possible workflow would be:

```
File Created/Modified
        ↓
File-System Monitor
        ↓
YARA Scanner
        ↓
Rule Matching
        ↓
Alert Generation
```

This would provide faster identification of suspicious artifacts without requiring the user to manually start a scan.

22.2 Scheduled Scanning

The project could be integrated with Windows Task Scheduler to perform scans automatically at predefined intervals.

For example:

- Daily scans.
- Weekly scans.
- Startup scans.
- Scheduled threat-hunting scans.

This would reduce the dependency on manual execution.

22.3 Expanded YARA Rule Library

The current project uses a relatively small set of customized rules.

Future versions could include a larger collection of carefully validated rules covering:

- Malware families.
- Suspicious scripts.
- Command-line indicators.
- Persistence mechanisms.
- Ransomware indicators.
- Credential-theft indicators.
- Web-shell indicators.
- Suspicious document characteristics.

A larger and better-maintained rule library would improve detection coverage.

22.4 Automated Rule Management

A future version could provide mechanisms for updating YARA rules from a trusted internal rule repository.

The system could periodically check for updated rules and deploy them after validation.

A validation stage should be included before activating new rules to reduce the risk of malformed rules or unexpected detections.

22.5 Graphical User Interface

The current project primarily uses PowerShell and command-line execution.

A graphical user interface could make the scanner easier to operate.

Possible GUI features include:

- Folder selection.
- Scan button.
- Progress indicator.
- Detection summary.
- Rule-match details.
- Scan history.
- Report viewer.
- Hash information.

22.6 Advanced File Hashing

The current project uses SHA-256 for evidence collection.

Future versions could calculate and compare multiple hashes, such as:

- SHA-256.
- SHA-1.
- MD5.

Hash values could be used as additional indicators for identifying previously analyzed files.

SHA-256 should remain the preferred modern cryptographic hash for integrity and evidence documentation.

22.7 Integration with Threat Intelligence

Future versions could integrate with approved threat-intelligence sources.

A detected file hash could be compared against known threat indicators to provide additional context.

The workflow could be:

```
YARA Detection
      ↓
SHA-256 Calculation
      ↓
Threat Intelligence Lookup
      ↓
Additional Context
      ↓
Analyst Investigation
```

Such integration would help analysts distinguish between known and previously unidentified artifacts.

22.8 SIEM Integration

The scanner could send detection events to a Security Information and Event Management (SIEM) platform.

Information such as the following could be forwarded:

- Detection rule.
- File path.
- Host information.
- Timestamp.

- SHA-256 hash.
- Severity.
- Investigation status.

This would allow YARA detections to be correlated with other security events.

22.9 EDR Integration

The project could be integrated with Endpoint Detection and Response (EDR) technologies.

YARA detections could provide additional file-level intelligence to an EDR platform, while EDR telemetry could provide behavioral context around the detected artifact.

This would create a more comprehensive detection workflow.

22.10 Automated Alerting

Future versions could generate alerts when suspicious indicators are detected.

Possible notification mechanisms include:

- Email notifications.
- Security dashboards.
- Centralized logging.
- SOC alert systems.

Alerts should contain enough information for analysts to begin an investigation without automatically performing destructive actions.

22.11 Quarantine Capability

A controlled quarantine feature could be introduced in a future version.

Instead of immediately deleting a detected file, the system could move it to a protected quarantine location after analyst approval.

A possible workflow would be:

```
Detection
   ↓
Analyst Validation
   ↓
Quarantine
   ↓
Hash Preservation
   ↓
Further Analysis
```

This approach would reduce the risk of permanently deleting potentially useful investigation evidence.

22.12 Improved Reporting

The current system generates text-based reports.

Future versions could generate more structured reports containing:

- Scan statistics.
- Detection severity.
- Rule names.
- File paths.
- SHA-256 hashes.
- Scan timestamps.
- Host information.
- Investigation status.

Reports could also be generated in formats such as CSV, JSON, HTML, or PDF.

22.13 Centralized Multi-Endpoint Scanning

The current implementation is designed primarily for a single Windows environment.

A future version could support multiple authorized endpoints from a centralized management system.

This would allow security teams to:

- Manage rules centrally.
- Schedule scans.
- Collect detection results.
- Compare endpoint results.
- Monitor multiple systems.

22.14 Improved Detection Correlation

Multiple YARA matches could be correlated to identify patterns across files and endpoints.

For example, repeated detections of the same indicator across multiple systems could receive a higher investigation priority.

This would help transform individual file detections into broader security intelligence.

22.15 Advanced Incident Response Workflow

The existing incident-response functionality could be expanded to include additional evidence collection, investigation tracking, and analyst workflows.

Future versions could record:

- File metadata.
- File hashes.
- Detection rules.
- Process information.
- Network indicators.
- Investigation timestamps.
- Analyst actions.

This would provide a more comprehensive incident-response capability.

22.16 Overall Future Scope

The long-term objective is to evolve the current proof-of-concept into a more complete security-analysis platform.

The proposed evolution can be represented as:

```
Current Project
      ↓
YARA + PowerShell
      ↓
Real-Time Monitoring
      ↓
Scheduled Scanning
      ↓
Threat Intelligence
      ↓
Centralized Reporting
      ↓
SIEM / EDR Integration
      ↓
Advanced Security Monitoring
```

These enhancements would improve the scalability, automation, detection coverage, and operational usefulness of the system while preserving its core YARA-based detection capability.

23. SECURITY CONSIDERATIONS

Security considerations are an important part of developing and operating a malware-detection system. Although the Simple Malware Scanner Using YARA is designed primarily for controlled detection and analysis, the scanner itself must be operated securely to prevent misuse, inaccurate results, loss of evidence, or unintended system changes.

The following security considerations were taken into account during the development and testing of the project.

23.1 Authorized Scanning

The scanner should only be used on systems, directories, and files for which the user has appropriate authorization.

Scanning unauthorized systems or data may violate organizational policies or applicable laws.

For this project, testing was performed within a controlled laboratory environment.

23.2 Controlled Test Artifacts

The project uses controlled test artifacts specifically designed to trigger the configured YARA rules.

These artifacts should not be interpreted as confirmed malware merely because a YARA rule matches them.

A YARA match means that the conditions specified by the rule were satisfied and should be treated as an indicator requiring further analysis.

23.3 Protection of YARA Rules

YARA rules represent detection logic and should be protected from unauthorized modification.

Unauthorized changes could:

- Disable detections.
- Introduce incorrect detection conditions.
- Increase false positives.
- Reduce detection coverage.

Therefore, rule files should be stored in protected locations and changes should be reviewed before deployment.

23.4 Safe Script Execution

The PowerShell scripts used by the project should be executed from trusted project locations.

Before executing a script, the analyst should verify that its contents have not been unexpectedly modified.

This is particularly important when scripts are transferred between systems.

23.5 Evidence Integrity

Incident-response information should be preserved carefully.

The project uses SHA-256 hashing to provide an integrity-related identifier for collected files.

A SHA-256 value can be recorded alongside other evidence so that the same file can be identified during subsequent analysis.

23.6 Protection of Generated Reports

The project generates reports containing information about detected artifacts and file locations.

These reports should be protected from unauthorized access or modification because they may contain useful security-investigation information.

Recommended controls include:

- Appropriate file-system permissions.
- Secure storage.
- Controlled access.
- Backup of important reports.
- Protection against unauthorized modification.

23.7 Avoiding Automatic Destructive Actions

The current scanner does not automatically delete detected files.

This is an intentional security consideration.

A YARA match does not necessarily mean that a file is malicious. Automatically deleting every matching file could result in:

- Loss of legitimate files.
- Loss of investigation evidence.
- System instability.

- Operational disruption.

Therefore, detection should normally be followed by validation before remediation.

23.8 False Positive Handling

Security analysts should investigate unexpected detections.

If a legitimate file triggers a rule, the rule may require refinement.

A suitable workflow is:

```
YARA Match
    ↓
Analyst Review
    ↓
File Validation
    ↓
Rule Assessment
    ↓
Confirmed Detection / False Positive
```

This reduces the possibility of inappropriate security actions.

23.9 False Negative Considerations

The absence of a YARA match should not be considered proof that a file is completely safe.

YARA detection depends on the indicators represented by the configured rules.

Therefore, important investigations should use additional security controls where appropriate.

23.10 Secure Rule Development

YARA rules should be tested against both matching and clean artifacts before being introduced into operational environments.

Testing should evaluate:

- Expected detections.
- False positives.
- Rule performance.
- Rule syntax.
- Detection coverage.

This helps maintain reliable detection logic.

23.11 Principle of Least Privilege

The scanner should operate with the minimum privileges necessary to perform its intended task.

Running scripts with unnecessary administrative privileges can increase the potential impact of an accidental or malicious modification.

Where elevated privileges are required for a particular authorized scan, they should be used only for the required operation.

23.12 Secure Handling of Suspicious Files

Detected files should be handled carefully during analysis.

Analysts should avoid unnecessarily executing suspicious artifacts on production systems.

When deeper analysis is required, isolated laboratory environments or appropriate malware-analysis infrastructure should be used.

23.13 System Availability

Large-scale scanning can consume CPU, memory, and disk resources.

In production environments, scanning schedules and scope should therefore be planned to minimize operational impact.

For example, organizations may schedule intensive scans during appropriate maintenance or low-usage periods.

23.14 Logging and Auditing

A production version of the system should maintain sufficient logs to record:

- Scan start time.
- Scan completion time.
- Target location.
- Number of files scanned.
- Number of matches.
- Matching rules.
- File paths.
- Hash values.
- Errors encountered.

Logging improves accountability and supports incident investigation.

23.15 Secure Future Integrations

If future versions integrate the scanner with external systems such as SIEM, EDR, threat-intelligence platforms, or centralized management systems, secure authentication and communication should be implemented.

Sensitive information should not be transmitted through insecure channels.

23.16 Security Assessment

The security considerations discussed above help ensure that the scanner is used responsibly and that its results are interpreted correctly.

The most important principles for the current project are:

1. Scan only authorized systems.
2. Use controlled test artifacts during laboratory testing.
3. Protect YARA rules from unauthorized modification.
4. Preserve investigation evidence.
5. Protect generated reports.
6. Validate detections before remediation.
7. Avoid unnecessary administrative privileges.
8. Do not execute suspicious files on production systems.
9. Maintain appropriate logging.
10. Combine YARA with additional security controls for comprehensive protection.

23.17 Overall Security Consideration

The Simple Malware Scanner Using YARA should be considered a security-analysis component rather than a complete endpoint-protection solution.

Its effectiveness is maximized when YARA detection is combined with appropriate security practices, analyst validation, endpoint monitoring, behavioral analysis, and other defensive technologies.

The secure operation of the scanner is therefore as important as the detection capability itself.

24. CONCLUSION

The **Simple Malware Scanner Using YARA** project was successfully designed and implemented as a lightweight, customizable, and automated file-detection solution. The primary objective of the project was to demonstrate how YARA's rule-based pattern-matching capabilities can be combined with PowerShell automation to identify controlled suspicious indicators within files and directories.

The project successfully implemented custom YARA rules for detecting predefined indicators. Individual file scanning was performed to verify the detection logic, while clean-file testing was used to demonstrate that files without matching indicators were not reported by the configured rules.

Recursive scanning was also implemented successfully. The scanner was able to examine nested directories and identify a controlled test artifact located several directory levels below the main scanning directory. This demonstrated that the system can search beyond a single directory level.

PowerShell automation further extended the functionality of the project. The main scanner automatically performed recursive scanning, calculated scan statistics, displayed detection results, and generated a report. Additional scripts were developed for endpoint security scanning, threat hunting, and incident-response activities.

The final scanner test processed six controlled files and generated five YARA rule matches in approximately 0.056 seconds. The results demonstrated that the implemented detection workflow operated successfully within the project's controlled testing environment.

The project also incorporated SHA-256 evidence collection as part of the incident-response workflow. Hash values provide useful file-identification and integrity-related information that can support subsequent security investigation.

The project demonstrates several important cybersecurity concepts, including:

- Rule-based malware detection.
- YARA rule development.
- Pattern matching.
- Recursive file scanning.
- PowerShell automation.
- Endpoint security scanning.
- Threat hunting.
- Incident response.
- SHA-256 hashing.

- Security report generation.

An important conclusion from the project is that a YARA match should be treated as a detection indicator rather than automatic proof that a file is malicious. The effectiveness of YARA-based detection depends on the quality, specificity, and coverage of the rules. False positives and false negatives are therefore possible, and detections should be validated using appropriate investigation techniques.

The current implementation is a proof-of-concept designed for controlled cybersecurity training and demonstration. It does not attempt to replace enterprise antivirus, EDR, SIEM, or other comprehensive security platforms. Instead, it demonstrates how YARA can serve as a flexible detection component within a broader security workflow.

Future improvements could include real-time file monitoring, scheduled scanning, expanded rule libraries, graphical interfaces, threat-intelligence integration, centralized reporting, SIEM/EDR integration, automated alerting, and controlled quarantine capabilities.

Overall, the project successfully achieved its primary objective of developing and demonstrating a **Simple Malware Scanner Using YARA**. It provides a practical foundation for understanding rule-based detection and demonstrates how lightweight security tools can be combined with automation to support malware analysis, endpoint security, threat hunting, and incident-response activities.

25. REFERENCES

The following references were consulted for understanding YARA, PowerShell, cryptographic hashing, malware detection, and cybersecurity concepts used during the development of the project.

25.1 YARA Documentation

VirusTotal. *YARA Documentation and User Guide*.
Official documentation covering YARA installation, rule syntax, command-line usage, scanning, and rule development.

25.2 YARA GitHub Repository

VirusTotal. *YARA – The pattern matching swiss knife for malware researchers*.
The official source repository containing the YARA project, source code, documentation, and releases.

25.3 Microsoft PowerShell Documentation

Microsoft. *PowerShell Documentation*.
Official Microsoft documentation covering PowerShell commands, scripting, execution, file handling, automation, and Windows administration.

25.4 Microsoft PowerShell Security Documentation

Microsoft. *about_Execution_Policies*.
Documentation describing PowerShell execution policies and their role in controlling script execution.

25.5 SHA-256 / Secure Hash Standard

National Institute of Standards and Technology (NIST). *Secure Hash Standard (SHS)*, Federal Information Processing Standards Publication 180-4.

This reference was used for understanding cryptographic hash functions and SHA-256.

25.6 NIST Cybersecurity Framework

National Institute of Standards and Technology (NIST). *Cybersecurity Framework (CSF)*.

The framework provides guidance for managing cybersecurity risks and supports concepts related to detection, response, and recovery.

25.7 NIST Computer Security Incident Handling Guide

National Institute of Standards and Technology (NIST). *Computer Security Incident Handling Guide*, Special Publication 800-61.

This reference supports the incident-response concepts discussed in the project.

25.8 MITRE ATT&CK

MITRE. *MITRE ATT&CK Framework*.

The framework provides information about adversary tactics, techniques, and procedures and can support threat-hunting and detection-engineering activities.

25.9 OWASP

Open Worldwide Application Security Project (OWASP). *OWASP Resources*.

OWASP provides cybersecurity guidance, security testing resources, and educational material relevant to application and information security.

25.10 Project Documentation and Test Evidence

The following project-generated resources were also used as implementation evidence:

- Custom YARA rule files.
- PowerShell scanner scripts.
- Controlled test artifacts.
- Scanner output.
- Endpoint scanning report.
- Threat-hunting report.
- Incident-response report.
- SHA-256 evidence generated during testing.
- Project screenshots and terminal output.

25.11 Reference Usage

The references listed above were used to support the technical concepts and implementation decisions presented throughout this project.

The actual experimental results, screenshots, YARA rules, PowerShell scripts, and generated reports included in this document were produced as part of the project's own controlled implementation and testing activities.

26. APPENDIX

The appendix contains the main source code and YARA rule files developed and used during the implementation of the Simple Malware Scanner Using YARA project.

The purpose of this appendix is to provide the complete implementation details and allow the project functionality to be reviewed, reproduced, and understood.

The appendix is divided into the following subsections:

- **29.1 PowerShell Scanner Script**
- **29.2 Endpoint Scanner Script**
- **29.3 Incident Response Script**
- **29.4 Threat Hunting Script**
- **29.5 Sample YARA Rules**

The scripts and rules presented in this appendix correspond to the implementation used during the project's controlled testing activities.

26.1 POWERSHELL SCANNER SCRIPT

The following PowerShell script implements the main automated malware-scanning functionality of the project.

The script invokes the YARA executable, recursively scans the configured sample directory, stores the complete results, counts rule matches, identifies matched files, and displays a scan summary.

Source Code

```
# Simple Malware Scanner Using YARA

$YaraPath = "C:\Users\ubaid\YARA\yara64.exe"
$RulesPath = "C:\Users\ubaid\YARA-Project\rules\master_rules.yar"
$SamplesPath = "C:\Users\ubaid\YARA-Project\samples"
$ResultsPath = "C:\Users\ubaid\YARA-Project\results\scan_results.txt"

Write-Host "========================================"
Write-Host " SIMPLE MALWARE SCANNER USING YARA"
Write-Host "========================================"
Write-Host ""

$StartTime = Get-Date

Write-Host "Scan Started : $StartTime"
Write-Host "Target Folder: $SamplesPath"
Write-Host ""

# Run YARA recursively
$Results = & $YaraPath -r $RulesPath $SamplesPath
```

```
# Save complete results
$Results | Out-File -FilePath $ResultsPath

# Count detections
$DetectionCount = $Results.Count

# Extract unique files
$MatchedFiles = @()

foreach ($Result in $Results) {
    if ($Result -match "^(.*?) (C:\\.*)$") {
        $MatchedFiles += $Matches[2]
    }
}

$UniqueFiles = $MatchedFiles | Sort-Object -Unique

$EndTime = Get-Date
$Duration = ($EndTime - $StartTime).TotalSeconds

Write-Host ""
Write-Host "========== SCAN SUMMARY =========="
Write-Host "Total Files Scanned : 6"
Write-Host "Rule Matches        : $DetectionCount"
Write-Host ("Scan Duration       : {0:N3} Seconds" -f $Duration)
Write-Host "================================="
Write-Host ""

Write-Host "Detection Results:"
$Results | ForEach-Object {
    Write-Host $_
}

Write-Host ""
Write-Host "Full results saved to:"
Write-Host $ResultsPath
```

Script Functionality

The script performs the following operations:

1. Defines the YARA executable path.
2. Defines the master YARA rule path.
3. Defines the sample directory.
4. Defines the output report location.
5. Starts the scan timer.
6. Executes YARA recursively.
7. Saves the scan output.
8. Counts rule matches.

9. Identifies unique matched files.
10. Calculates scan duration.
11. Displays the final scan summary.

26.2 ENDPOINT SCANNER SCRIPT

The endpoint scanner extends the main YARA scanning functionality to selected user directories. It was designed to demonstrate how YARA rules can be applied to common endpoint locations such as Downloads, Desktop, and Documents.

Source Code

```
# Endpoint Security Scanner Using YARA

$YaraPath = "C:\Users\ubaid\YARA\yara64.exe"
$RulesPath = "C:\Users\ubaid\YARA-Project\rules\master_rules.yar"
$ResultsPath = "C:\Users\ubaid\YARA-Project\results\endpoint_report.txt"

$Locations = @(
    "$env:USERPROFILE\Downloads",
    "$env:USERPROFILE\Desktop",
    "$env:USERPROFILE\Documents"
)

$AllResults = @()

foreach ($Location in $Locations) {

    Write-Host ""
    Write-Host "Scanning: $Location"

    if (Test-Path $Location) {

        $Results = & $YaraPath -r $RulesPath $Location

        if ($Results) {
            Write-Host "Suspicious indicators found."
            $AllResults += $Results
        }
        else {
            Write-Host "No suspicious indicators found."
        }
    }
    else {
        Write-Host "Location not found."
    }
}

$AllResults | Out-File -FilePath $ResultsPath

Write-Host ""
Write-Host "Endpoint report saved to:"
Write-Host $ResultsPath
```

Script Functionality

The endpoint scanner:

- Defines the YARA executable.
- Loads the master YARA rules.
- Defines selected endpoint locations.
- Recursively scans each location.
- Displays whether indicators were found.
- Collects the detection results.
- Saves the results to `endpoint_report.txt`.

26.3 INCIDENT RESPONSE SCRIPT

The incident-response script supports basic evidence collection from detected artifacts. It records relevant file information and calculates SHA-256 hashes that can assist with file identification during investigation.

Source Code

```
# Incident Response Evidence Collection

$ResultsPath = "C:\Users\ubaid\YARA-Project\results\scan_results.txt"
$IncidentReport = "C:\Users\ubaid\YARA-Project\results\incident_report.txt"

"========================================" | Out-File $IncidentReport
"       INCIDENT RESPONSE REPORT" | Out-File $IncidentReport -Append
"========================================" | Out-File $IncidentReport -
Append
"" | Out-File $IncidentReport -Append

if (Test-Path $ResultsPath) {

    $Results = Get-Content $ResultsPath

    foreach ($Result in $Results) {

        if ($Result -match "^(.*?) (C:\\.*)$") {

            $Rule = $Matches[1]
            $FilePath = $Matches[2]

            if (Test-Path $FilePath) {

                $Hash = Get-FileHash -Path $FilePath -Algorithm SHA256

                "Detection Rule : $Rule" |
                    Out-File $IncidentReport -Append

                "File Path      : $FilePath" |
                    Out-File $IncidentReport -Append
```

```
                "SHA-256        : $($Hash.Hash)" |
                    Out-File $IncidentReport -Append

                "Timestamp      : $(Get-Date)" |
                    Out-File $IncidentReport -Append

                "----------------------------------------" |
                    Out-File $IncidentReport -Append
            }
        }
    }
}

Write-Host ""
Write-Host "Incident response report saved to:"
Write-Host $IncidentReport
```

Script Functionality

The script:

1. Reads the scanner's detection results.
2. Extracts the matching rule and file path.
3. Verifies that the file exists.
4. Calculates its SHA-256 hash.
5. Records the detection information.
6. Stores the evidence in `incident_report.txt`.

The collected information can support subsequent investigation and documentation.

26.4 THREAT HUNTING SCRIPT

The threat-hunting script recursively searches selected simulated endpoint locations using the master YARA rule set.

The project uses the following threat-hunting structure:

```
ThreatHunt
├── Documents
├── Downloads
├── Temp
└── USB
```

Source Code

```
# YARA Threat Hunting Script

$YaraPath = "C:\Users\ubaid\YARA\yara64.exe"
$RulesPath = "C:\Users\ubaid\YARA-Project\rules\master_rules.yar"
$ThreatHuntPath = "C:\Users\ubaid\YARA-Project\ThreatHunt"
```

```
$ReportPath = "C:\Users\ubaid\YARA-
Project\results\threat_hunting_report.txt"

$Results = @()

Write-Host "========================================"
Write-Host "          YARA THREAT HUNTING"
Write-Host "========================================"
Write-Host ""

if (Test-Path $ThreatHuntPath) {

    Write-Host "Threat Hunt Location:"
    Write-Host $ThreatHuntPath
    Write-Host ""

    $Results = & $YaraPath -r $RulesPath $ThreatHuntPath

    if ($Results) {
        Write-Host "Suspicious indicators found."
    }
    else {
        Write-Host "No suspicious indicators found."
    }

    $Results | Out-File -FilePath $ReportPath
}
else {
    Write-Host "Threat hunting directory not found."
}

Write-Host ""
Write-Host "Threat hunting report saved to:"
Write-Host $ReportPath
```

Script Functionality

The script:

- Defines the threat-hunting directory.
- Loads the master YARA rules.
- Recursively scans the simulated locations.
- Displays detection status.
- Saves the results to `threat_hunting_report.txt`.

This demonstrates how YARA can be used as part of a basic proactive threat-hunting workflow.

26.5 SAMPLE YARA RULES

The following sample rules were created for the controlled testing environment of the project.

These rules are intended to demonstrate YARA rule development and pattern matching. The test artifacts used with these rules are controlled samples designed to trigger the corresponding detection conditions.

26.5.1 Test Malware Detection Rule

```
rule Test_Malware_Detection
{
    meta:
        description = "Detects a controlled test indicator"
        author = "Project"
        purpose = "Laboratory testing"

    strings:
        $indicator = "MALWARE_TEST"

    condition:
        $indicator
}
```

26.5.2 Suspicious PowerShell Indicator

```
rule Suspicious_PowerShell_Indicator
{
    meta:
        description = "Detects a controlled PowerShell indicator"
        author = "Project"
        purpose = "Laboratory testing"

    strings:
        $powershell = "powershell"

    condition:
        $powershell
}
```

26.5.3 Suspicious Command Indicator

```
rule Suspicious_Command_Indicator
{
    meta:
        description = "Detects a controlled command indicator"
        author = "Project"
        purpose = "Laboratory testing"

    strings:
        $command = "cmd"

    condition:
        $command
}
```

26.5.4 Suspicious Script Indicator

```
rule Suspicious_Script_Indicator
{
    meta:
```

```
        description = "Detects a controlled scripting indicator"
        author = "Project"
        purpose = "Laboratory testing"

    strings:
        $script = "script"

    condition:
        $script
}
```

26.5.5 Master Rule File

The master rule file combines the project detection rules into a single rule set used by the automated scanner.

The master rule concept allows the scanner to execute one YARA command against a consolidated collection of detection rules.

Rule Development Summary

The sample rules demonstrate the fundamental components of a YARA rule:

```
Rule Name
    ↓
Metadata
    ↓
Strings / Indicators
    ↓
Condition
    ↓
Detection Match
```

These rules were designed specifically for controlled laboratory testing. A match indicates that the configured rule condition was satisfied and should not, by itself, be interpreted as confirmation that an artifact is actual malware.